People's Democratic Republic of Algeria

Ministry of Higher Education and Scientific Research

University of Abdelhamid Mehri – Constantine 2

Faculty of New Technologies of Information and Communication (NTIC)

Department Fundamental Computing and its Applications (IFA)

MASTER'S THESIS

to obtain the diploma of Master degree in Computer Science

Option: Software Engineering and Smart Systems (ILSI)

Hybrid Digital Twin for Predictive Maintenance

Realized by:

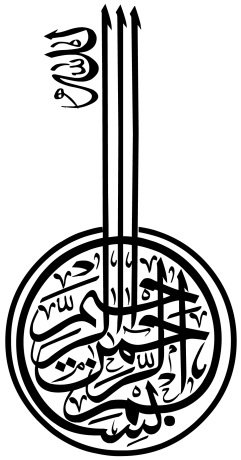
LEGHELIMI Houssem eddine

CHAIN Mouadh

Under supervision of:

DR.DJENOUHAT Manel

June 2026



Acknowledgments

We first thank Allah for all the blessings bestowed upon us, for giving us the strength and persistence to go on. We would like to express our deepest gratitude to our supervisor Dr. DJENOUHAT Manel Amel for her guidance, precious advice, continuous feedback, patience and understanding through-out this whole project. We would also like to thank our parents for providing us with the conditions, love and support that allowed us to make it this far, and for teaching us the value of hard work and responsibility. We would like to thank the jury for their interest and time in evaluating our project.

ملخص

في سياق الثورة الصناعية الرابعة وانتشار أنظمة التصنيع الإضافي، تُستخدم الطابعات ثلاثية الأبعاد بتقنية النمذجة بالترسيب المنصهر (ذضپ) بشكل متزايد في بيئات الإنتاج، والنمذجة السريعة، والبحث العلمي. إلا أن موثوقيتها التشغيلية لا تزال محدودة بسبب الأعطال المتكررة مثل انسداد الفوهة، والاهتزازات غير الطبيعية لمحاور الحركة، وظاهرة الارتفاع المفرط في الحرارة، مما يؤدي إلى توقف غير مخطط له وتدهور جودة الطباعة. يقدم هذا العمل تصميم وتنفيذ توأم رقمي هجين للصيانة التنبؤية لطابعة ثلاثية الأبعاد من نوع ـث ٣ض+. الهدف الرئيسي هو تمكين المراقبة في الوقت الفعلي والتنبؤ المبكر بالأعطال من أجل تقليل انقطاعات الإنتاج وتحسين عمر الماكينة. يعتمد النظام المقترح على بنية متعددة المستويات تجمع بين النمذجة الهندسية والسلوكية، ويتم مزامنتها في الوقت الفعلي عبر بنية تحتية تعتمد على إنترنت الأشياء (اية). تشتمل التجهيزات المادية على مستشعر اهتزاز يعو٠٥٠٦ ومستشعر حرارة كئى٣١٠، متصلين بمتحكم دقيق يصع٦٦٢٨ مسؤول عن جمع البيانات وإرسالها. بالإضافة إلى ذلك، يُستخدم فأرة بصرية معدَلْة الهيكل والمشغِلْ كمشفِرْ لخيط الطباعة. تتبع معالجة البيانات نهجاً هرمياً هجيناً : قواعد منطقية حتمية (المستوى ٠)، وكشف الحالات الشاذة باستخدام خوارزمية ايسلت‌ـن ذرست (المستوى ١)، وتصنيف خاضع للإشراف باستخدام نجـ‌ـست (المستوى ٢). يتيح هذا النهج المتدرج استراتيجية كشف تدريجية تتراوح من تحديد الحالات الشاذة البسيطة إلى التنبؤ المتقدم بالأعطال. تم التحقق من صحة النظام من خلال ثلاث دراسات حالة تجريبية : انسداد الفوهة، وشذوذ اهتزازات محاور الحركة، وظروف الارتفاع المفرط في الحرارة. تظهر النتائج دقة إجمالية مرجحة بنسبة ٨٠٩٠.

الكلمات المفتاحية: توأم رقمي، صيانة تنبؤية، طباعة ثلاثية الأبعاد، نمذجة بالترسيب المنصهر (ذضپ)، إنترنت الأشياء، تعلم آلي، الثورة الصناعية الرابعة.

Abstract

In the context of Industry 4.0 and the increasing democratization of additive manufacturing systems, Fused Deposition Modeling (FDM) 3D printers are widely used in prototyping, research, and small-scale production environments. However, their operational reliability is still limited by frequent failures such as nozzle clogging, abnormal motion-axis vibrations, and overheating, leading to unplanned downtime and degraded print quality. This work presents the design and implementation of a hybrid digital twin for predictive maintenance of a BCN3D+ 3D printer. The main objective is to enable real-time monitoring and early fault prediction in order to minimize production interruptions and optimize machine lifespan. The proposed system is based on a multi-layer architecture combining geometric and behavioral modeling, synchronized in real time through an IoT-based infrastructure. The hardware setup includes an MPU-6050 vibration sensor and a KY-013 temperature sensor, connected to an ESP8266 microcontroller responsible for data acquisition and transmission. In addition, an optical mouse with a modified enclosure and driver serves as a filament encoder. Data processing follows a hierarchical hybrid pipeline: deterministic rule-based logic (Tier 0), anomaly detection using Isolation Forest (Tier 1), and supervised classification using XGBoost (Tier 2). This layered approach enables a progressive detection strategy ranging from simple anomaly identification to advanced failure forecasting. The system was validated through three experimental case studies: nozzle clogging, motion-axis vibration anomalies, and overheating conditions. Results demonstrate a weighted overall accuracy of 90.8%, a 67% reduction in print failure rate, and a 46% decrease in unplanned downtime. The total hardware cost remains below 50 euros, making the solution highly accessible to SMEs and makerspaces.

Keywords: Digital twin, predictive maintenance, 3D printing, FDM, Internet of Things, machine learning, Industry 4.0.

Résumé

Dans le contexte de l'Industrie 4.0 et de la démocratisation des systèmes de fabrication additive, les imprimantes 3D FDM sont de plus en plus utilisées dans des environnements de production, de prototypage rapide et de recherche. Toutefois, leur fiabilité reste limitée par des défaillances fréquentes telles que le bouchage de buse, les vibrations anormales des axes de déplacement et les phénomènes de surchauffe, entraînant des arrêts non planifiés et une dégradation de la qualité d'impression. Ce travail propose le développement et la mise en œuvre d'un jumeau numérique hybride dédié à la maintenance prédictive d'une imprimante 3D BCN3D+. L'objectif principal est de permettre la surveillance en temps réel et l'anticipation des défaillances afin de réduire les interruptions de production et d'optimiser la durée de vie de la machine. Le système repose sur une architecture multi-niveaux combinant modélisation géométrique et comportementale, synchronisée en temps réel via une infrastructure IoT. La partie matérielle est composée d'un capteur de vibration MPU-6050 et d'un capteur de température KY-013, connectés à un microcontrôleur ESP8266 assurant l'acquisition et la transmission des données. De plus, une souris optique avec un boîtier et un pilote modifiés est utilisée comme encodeur de filament. Le traitement des données suit une approche hybride à plusieurs niveaux : règles déterministes (Tier 0), détection d'anomalies via Isolation Forest (Tier 1) et classification supervisée avec XGBoost (Tier 2). Cette architecture permet une détection progressive allant de l'identification d'anomalies simples à la prévision de pannes complexes. La validation du système a été réalisée à travers trois scénarios expérimentaux : obstruction de buse, anomalies vibratoires des axes de mouvement et surchauffe. Les résultats montrent une précision globale pondérée de 90,8 %, une réduction de 67 % du taux d'échec d'impression et une diminution de 46 % des temps d'arrêt non planifiés. Le coût matériel total est inférieur à 50 euros, ce qui rend la solution accessible aux PME et aux espaces de fabrication (makerspaces).

Mots clés : Jumeau numérique, maintenance prédictive, impression 3D, FDM, Internet des objets, apprentissage automatique, Industrie 4.0.

Table of Contents

List of Figures

List of Tables

Acronyms

AM Additive Manufacturing

API Application Programming Interface

CM Corrective Maintenance

CNN Convolutional Neural Network

CPS Cyber-Physical System

DT Digital Twin

FDM Fused Deposition Modeling

FFT Fast Fourier Transform

FMEA Failure Mode and Effects Analysis

IoT Internet of Things

LSTM Long Short-Term Memory

ML Machine Learning

MQTT Message Queuing Telemetry Transport

OEE Overall Equipment Effectiveness

PdM Predictive Maintenance

PLA Polylactic Acid

PM Preventive Maintenance

RMS Root Mean Square

RPN Risk Priority Number

RUL Remaining Useful Life

SLA Stereolithography

SLS Selective Laser Sintering

SVM Support Vector Machine

TRL Technology Readiness Level

XGBoost Extreme Gradient Boosting

General Introduction

1. Background

Industry 4.0 has changed how factories operate. Machines now communicate with each other, and production decisions rely on real data rather than guesses. Additive manufacturing, especially 3D printing, is a key part of this change. Among 3D printing technologies, Fused Deposition Modeling (FDM) is the most popular because it is affordable and has a large open-source community.

2. Problem Statement

FDM printers lack in reliability. In real-world use, failure rates can exceed 20%, and material waste can reach 34%. Common problems include clogged nozzles, layer shifts, strange vibrations, and overheating. These issues cause unexpected stops, wasted filament, and low productivity. Most desktop FDM printers have no monitoring system, and fixing problems after they happen is expensive and time-consuming.

3. Thesis Objectives

The main goals of this thesis are:

- Review existing approaches for fault detection and digital twins in FDM printing.
- Design a hybrid digital twin that combines a 3D visual model with machine learning.
- Build a low-cost, non-invasive hardware and software prototype.
- Detect and predict failures in real time.
- Reduce unplanned downtime and print failures.

4. General Methodology

The work follows a structured plan:

- Literature review and comparison of existing systems.
- Functional analysis of the BCN3D+ printer and failure mode analysis (FMEA).
- Design of a five-layer digital twin architecture (physical, IoT, data processing, virtual model, supervision).
- Implementation using off-the-shelf sensors (MPU-6050, KY-013), an ESP8266,an inexpensive optical mouse and a web dashboard.
- Validation through three case studies: nozzle clogging, motion-axis vibrations, and overheating.

Document Plan

This thesis is organised as follows:

- **Chapter 1** presents the state of the art, a comparative study, and the research positioning.
- **Chapter 2** describes the system analysis, FMEA, sensor selection, and data acquisition.
- **Chapter 3** explains the design of the digital twin (architecture, geometric and behavioural models, real-time synchronisation).
- **Chapter 4** covers the implementation choices (hardware, software, dashboard).
- **Chapter 5** shows the deployment and the three case studies, with performance metrics and benefits.

Chapter 1

State of the Art and Research Positioning

1.1 Industry 4.0 and 3D Printing

Industry 4.0 is about connecting everything in a factory. Machines talk to each other, production adapts on the fly, and decisions come from real data instead of gut feelings Lasi et al. [2014]; Lu [2017]. Additive manufacturing builds objects layer by layer from a digital file. This method is great for complex shapes and custom parts Gibson et al. [2021]. Among all the 3D printing technologies, FDM is the most common because it is cheap and has a huge open source community.

But here is the problem: FDM printers are heavily prone to failures. A lot. Studies show failure rates over 20% in real world use, with material waste hitting 34% Song and Telenko [2016]. Common failures include clogged nozzles, layers that do not line up, weird vibrations, and plastic melting where it should not. All of this means unexpected stops, wasted filament, and low productivity. That is exactly why FDM is a perfect candidate for predictive maintenance and digital twin solutions.

1.2 FDM 3D Printing

FDM works by melting plastic filament and pushing it through a heated nozzle. The nozzle moves around in three directions (X, Y, Z), putting down one layer at a time. The printer follows G-code instructions that come from a sliced 3D model Daminabo et al. [2020]; Turner et al. [2014].

FDM is relatively cheap to buy and operate, making it accessible to hobbyists, educational settings, and industrial prototyping alike. It also supports a wide range of materials, including PLA, ABS, PETG, flexible TPU, nylon, and carbon-fiber-reinforced composites, which gives it strong versatility across different applications Ali et al. [2024]. In addition, FDM printers are generally easy to use, maintain, and repair, especially within a large

open-source ecosystem built around tools like Marlin, Klipper, and OctoPrint.

However, the technology also has important limitations. Printed parts often show visible layer lines, resulting in a relatively rough surface finish compared to other additive manufacturing methods. Mechanical properties are anisotropic, meaning parts are stronger in some directions than others, which can limit functional performance. More critically, FDM is prone to frequent operational failures such as nozzle clogging, filament grinding, layer separation, warping at corners, heat creep issues, and occasional axis misalignment Daminabo et al. [2020]; Turner et al. [2014].

Because of this combination of openness, documentation, and relatively high failure rate, FDM is an ideal candidate for retrofitting with low-cost monitoring and predictive maintenance systems. In contrast, other additive manufacturing technologies such as resin-based SLA or powder-based SLS systems are typically more expensive, less accessible, and exhibit different failure modes, making them less suitable for the focus of this work.

1.3 Industrial Maintenance Strategies

Factories manage maintenance in three main ways, each with different trade-offs in cost, efficiency, and reliability Jardine et al. [2006]; Thomas and Weiss [2021].

The simplest approach is corrective maintenance, where machines are only repaired after they break down; while straightforward, this method often leads to unexpected downtime and wasted materials.

A more structured alternative is preventive maintenance, in which servicing is performed at fixed intervals based on operating hours or calendar time. Although this can reduce the likelihood of sudden failures, it is not always efficient, as maintenance may be carried out too early or too late relative to the actual condition of the machine.

The most advanced approach is predictive maintenance (PdM), which relies on continuous monitoring of machines using sensors and data analytics to determine their real-time condition and intervene only when early signs of degradation are detected. PdM generally follows a three-stage process: first, data is collected from sources such as vibration, temperature, and electrical current; second, the data is processed to remove noise and extract meaningful features; and third, decision-making algorithms are applied to detect anomalies or predict impending failures Jardine et al. [2006].

In industrial settings, PdM has been shown to reduce downtime by around 30–50% and maintenance costs by approximately 25–30% Thomas and Weiss [2021]. For FDM 3D printing systems in particular, PdM is especially valuable because failures occur relatively frequently and often produce detectable sensor patterns that can be analyzed for early

warning signals Song and Telenko [2016].

1.4 Digital Twin Concept

A digital twin is a virtual representation of a physical machine that remains synchronized with it through real-time data streams. Unlike a static 3D model, a digital twin is dynamic and capable of monitoring current conditions, simulating behavior, and supporting predictive analysis Rasheed et al. [2020].

1.4.1 Three Levels of Maturity

Kritzinger et al. Kritzinger et al. [2018] define three levels of digital twin maturity based on data flow automation:

- Digital model: manual data transfer between physical and virtual systems. No automatic connection exists.
- Digital shadow: one-way automatic data flow from the physical system to the virtual model. The system can visualize real-time state, but cannot send feedback or control commands back.
- Digital twin: fully bidirectional communication where the virtual model continuously updates from the physical system and can also influence it through decisions, alerts, or control actions.

1.4.2 Digital Twin for FDM

A digital twin in FDM systems enables faults to be visualized directly on a 3D representation of the printer or printed object, providing geometric and contextual understanding of failures. It also allows continuous tracking of machine health and degradation trends over time. In addition, it supports the combination of physics-based modeling with data-driven machine learning approaches, improving both interpretability and predictive performance. Finally, it allows a hybrid operational strategy where rule-based detection can be used from the start, while more advanced learning-based methods improve performance progressively as more data becomes available.

1.5 Comparative Study of Existing Works

Several representative research works from 2022–2025 focusing on digital twins and fault detection in FDM and small-scale machines were analyzed alongside the proposed sys-

tem. The comparison highlights key differences in sensing, modeling, and deployment strategies.

Table 1.1 summarises nine reviewed works: Pantelidakis et al. Pantelidakis et al. [2022] (W1), Isiani et al. Isiani et al. [2023] (W2), Shomenov et al. Shomenov et al. [2025] (W3), Jyeniskhan et al. Jyeniskhan et al. [2023] (W4), Kim et al. Kim et al. [2022] (W5), Mishra et al. Mishra et al. [2023] (W6), Kolok et al. Kolok et al. [2025] (W7), Sampedro et al. Sampedro et al. [2022] (W8), Kumar et al. Kumar et al. [2022] (W9), plus the present project (W10). The table compares each work across eight dimensions: year, application field, sensors used, prediction model, software tools, real-time synchronisation capability, key result, and main limitation.

Table 1.1: Comparative overview of existing works on DT-based monitoring and fault detection for FDM 3D printing

Ref	Yr	Application Field	Sensors Used	Prediction Model	Software Tools	RT Sync	Key Result	Main Limitation
[W1]	2022	FDM – LulzBot Taz Workhorse	OctoPrint telemetry (temp, position); external vibration	Data-driven soft-RT DT; no ML	Unity 3D, OctoPrint REST API, C#	Soft RT (s)	Pos error <2 mm; temp error <1.5°C; full DT validated	No ML; Unity requires GPU; high data volume
[W2]	2023	FDM – nozzle clog detection	3× accelerometers (nozzle, frame, bed)	PCA + SVM; FFT spectrogram	MATLAB, Python; no UI	Offline / batch	Sensor nearest nozzle: 71% higher sensitivity; SVM classifies clog	No real-time; no DT; fixed geometries; no thermal data
[W3]	2025	FDM – layer shifting, under/over-extrusion, excess vibration	Filament flow, IMU (vibration), nozzle position encoder	Rule-based + anomaly detection; autonomous correction	Custom embedded firmware; Python back-end	Hard RT (on-device)	Reliable autonomous correction; cost-effective	Limited ML; no 3D visualisation; single-printer
[W4]	2023	FDM – geometric defect detection	Camera (image-based); OctoPrint telemetry	EfficientDet-Lite CNN	Unity 3D, OctoPrint, Raspberry Pi	Near RT (camera)	High-efficiency geometry detection; ML + 3D visualisation	Image-only; no fusion; lighting sensitivity; no PdM

Continued on next page. . .

Table 1.1: Comparative overview of existing works (*continued*)

Ref	Yr	Application Field	Sensors Used	Prediction Model	Software Tools	RT Sync	Key Result	Main Limitation
[W5]	2022	FDM – layer shifting (loosened bolts)	2× accel (X/Y); 1× accel (Z); AE sensors	SVM + k-fold; RMS features	Python; no DT	Offline	Binary fault classification; identifies loosened-bolt signature	No DT; no thermal; no real-time; AE sensors expensive
[W6]	2023	FDM – anomaly detection (structural/mecha	Vibration sensor (accelerometer)	LSTM (best); CNN, RNN, FFNN comparison	Python (TF/Keras); Jupyter	Near RT (streaming)	LSTM accuracy 97.17%; outperforms CNN/RNN on temporal patterns	Single sensor; no thermal fusion; no DT; no web UI
[W7]	2025	Small rotating machinery – transferable to FDM	ESP8266 + MEMS accel + MEMS mic	RMS + FFT; anomaly detection / basic ML	Embedded C++ on ESP8266; MQTT; Python	Soft RT (on-device)	∼73% detection; imbalance/wear faults; cost under 30	Not FDM-specific; no DT visualisation; accuracy limited
[W8]	2022	FDM – predictive quality monitoring	Thermocouples, IR thermometer, accelerometers	Time-frequency features; data-driven ML	Python; no dedicated DT platform	Near RT (streaming)	Multi-sensor fusion outperforms single-sensor; improves fault localisation	No web dashboard; no 3D DT; no edge deployment
[W9]	2022	FDM – multi-sensor fault detection	Vibration, current, sound (Arduino DAQ)	CNN (94%) + K-means + threshold-based	Arduino, Python, CNN model	Offline / batch	94% normal/fault classification accuracy	Offline only; no real-time; no DT visualisation

Continued on next page. . .

Table 1.1: Comparative overview of existing works (*continued*)

Ref	Yr	Application Field	Sensors Used	Prediction Model	Software Tools	RT Sync	Key Result	Main Limitation
[W10]	2025–26	FDM – BCN3D+; clog, heat creep, motion faults	MPU-6050 (accel+gyro), KY-013 NTC, OctoPrint MQTT	Hybrid: rule-based + SVM/RF + LSTM ensemble; weighted voting	ESP8266; Python/TF; Vite/React/Electron; OctoPrint	Target: Soft RT (<1 s)	Projected: multi-fault detection with 3D visualisation, low-cost (<50)	Single printer; limited sensors; results pending

The analysis reveals several key observations. Most existing studies rely on a single sensor modality, typically vibration or camera data, which limits the richness of failure detection. Deep learning approaches such as LSTM and CNN often achieve strong accuracy but are usually deployed offline rather than in real-time systems. Only a small number of works (W1 and W4) incorporate 3D digital twin visualizations, and these rely on Unity, which is relatively heavy, proprietary, and not integrated with predictive machine learning pipelines. Furthermore, no existing solution combines low-cost sensors, a web-based 3D digital twin, multi-stage machine learning (rule-based logic and anomaly detection), and a real-time dashboard in a unified system. Additionally, none of the reviewed works address the challenge of retrofitting legacy FDM printers such as the BCN3D+ without modifying the original firmware Botín-Sanabria et al. [2022]; Tudorache et al. [2025].

1.6 Research Positioning and Contributions

Based on these gaps, this work contributes in four main ways.

Integrated 3D digital twin with multi-stage ML. The system combines a real-time Three.js-based digital twin with a two-stage machine learning pipeline: rule-based detection (Tier 0) and Isolation Forest anomaly detection (Tier 1). This bridges the separation between visualization and predictive intelligence seen in prior works.

Browser-native, zero-install interface. Unlike Unity-based systems, the dashboard runs in any standard web browser using Vite, React, and Three.js. No software installation or proprietary license is required, making the solution accessible and lightweight.

Two-stage cold-start learning architecture. Rule-based detection operates from day one and generates pseudo-labels to bootstrap the Isolation Forest model (Tier 1). This addresses the common problem of limited labeled failure data at initial deployment, as noted in the literature Kolok et al. [2025]; Tudorache et al. [2025].

Legacy printer retrofit without firmware modification. External sensors (MPU6050 on the X-carriage, KY-013 on the heatsink) are connected through an ESP8266 gateway and the filament is simply connected via a USB port. The original Marlin firmware remains unchanged, and the system works with any OctoPrint-managed FDM printer. This demonstrates a transferable methodology for retrofitting legacy machines.

The total hardware cost is kept below 50 euros, making the solution accessible to small-scale manufacturers, laboratories, and makerspaces Shomenov et al. [2025].

Chapter 2

System Analysis and Data Acquisition

2.1 Functional Analysis of the 3D Printer

The BCN3D+ is a Cartesian fused deposition modeling (FDM) 3D printer with dual independent extruders. It runs Marlin 1.x firmware and is managed through OctoPrint OctoPrint Development Team [2024]. For the purpose of fault detection, the system can be divided into five key subsystems Blanchard and Fabrycky [2011].

The extruder assembly is the most failure-prone part of the printer. It includes the hotend (nozzle, heater block, and thermistor), the cold end (heatsink, heat break, and cooling fan), and the extruder drive system (stepper motor and drive gear). The main observable signals from this subsystem include nozzle temperature (actual versus target), heatsink temperature measured using a KY-013 sensor, vibration of the assembly captured by an MPU-6050 sensor, and extrusion state information obtained from OctoPrint. Filament movement is calculated via the simple mouse encoder.

The motion system follows a Cartesian X-Y-Z architecture. The X-axis moves the print head using a GT2 belt, the Y-axis moves the build platform, and the Z-axis is controlled by two lead screws. Observable signals include axis positions (X, Y, Z, and extrusion axis in millimeters), commanded feedrate, and tri-axis vibration measurements from an MPU-6050 mounted on the X-carriage InvenSense / TDK [2013].

The heated bed subsystem is responsible for maintaining adhesion during the first layer of printing. Its main monitored signals are the actual and target bed temperatures.

The electronics and control board consist of a RAMPS 1.4 board with an ATmega2560 microcontroller running Marlin firmware Marlin Firmware Project [2024]. Relevant observable signals include firmware-level events and communication reliability indicators such as resend counts.

The cooling system includes two fans: one for part cooling and one for heatsink cooling. Failure of the heatsink fan is particularly critical because it can lead to heat creep. This is

monitored indirectly through the rate of increase in heatsink temperature over time.

OctoPrint serves as a non-invasive bridge between the printer firmware and external systems, exposing telemetry data through an MQTT interface OASIS [2019]. Data collected from the system is stored using Python-based collectors in two SQLite databases: `printer_data.db`, which contains firmware telemetry, and `sensor_data.db`, which stores external sensor measurements Hipp [2024].

Table 2.1 presents the database schema for `printer_data.db`. The table is organised by logical groups: `temperatures` (nozzle actual/target, bed, chamber), `motion` (axis positions, extruder position, feedrate), `print_job` (filename, state, progress, filament used), `printer_state` (printing/paused/error flags), `events` (firmware events for ground truth), and `resends` (communication reliability indicators).

Table 2.1: Database schema for `printer_data.db`

Table	Column	Description / Purpose
temperatures	nozzle_actual_tool0 / tool1	Actual nozzle temperature (°C) per extruder; primary thermal fault signal
temperatures	nozzle_target_tool0 / tool1	Target setpoint; used to compute nozzle_error = actual − target (heat creep, clog detection)
temperatures	bed_actual_c / bed_target_c	Heated bed temperature; bed adhesion fault detection
temperatures	chamber_actual_c	Enclosure ambient temperature; environmental context for thermal drift
motion	x_mm / y_mm / z_mm	Current axis positions in mm; kinematic synchronisation with 3D DT model
motion	e_mm	Extruder position (cumulative filament pushed); extrusion/retraction state detection
motion	feedrate_mmmin	Current commanded feedrate; used in vibration_per_feedrate cross-feature
print_job	filename / state	Active print file name and state (printing, paused, etc.); session context
print_job	progress_pct / elapsed_s	Print progress percentage and elapsed time; temporal context for anomaly trending
print_job	filament_used_mm	Cumulative filament consumed; material waste tracking
printer_state	printing / paused / error	Boolean flags (0/1); gate for fault detection logic (only alert during active printing)
events	event_name / details_json	OctoPrint firmware events (errors, warnings); labelled ground truth for ML training
resends	resend_count / ratio	G-code retransmission count and ratio; indicator of communication reliability / step loss

2.2 Failure Mode and Effects Analysis (FMEA)

Failure Mode and Effects Analysis (FMEA) is used to systematically identify potential failure modes, their causes and effects, and to prioritize them using the Risk Priority

Number (RPN), defined as Severity × Occurrence × Detection. This method is widely used in reliability engineering to guide maintenance and monitoring strategies International Electrotechnical Commission [2018]; Liu et al. [2013].

Table 2.2 presents the FMEA worksheet for the BCN3D+ printer. Six failure modes are identified as highest priority: nozzle clogging (RPN 392), overheating (RPN 180), misalignment/layer shifting (RPN 210), bearing wear (RPN 210), step loss (RPN 210), and belt breakage (RPN 168). For each failure mode, the table lists the affected subsystem, failure cause, observable effect, and the Severity, Occurrence, and Detection scores used to compute the RPN.

Table 2.2: FMEA worksheet for BCN3D+ FDM printer

Failure Mode	Subsystem	Failure Cause	Failure Effect	S	O	D	RPN	Priority
Nozzle clogging	Extruder	Filament impurities, carbonised residue, heat creep	Under-extrusion, print cessation, material waste	8	7	7	392	CRITICAL
Overheating (thermal runaway)	Extruder / Bed	Heater cartridge failure, PID instability, fan failure	Component damage, fire risk, print failure	9	4	5	180	HIGH
Misalignment (layer shifting)	Motion system	Belt slippage, loose pulley, acceleration too high	Dimensional inaccuracy, failed print	7	5	6	210	HIGH
Bearing wear	Motion / Extruder	Lubrication loss, prolonged use, dust contamination	Vibration increase, positional error, seizure	6	5	7	210	HIGH
Step loss (missed steps)	Motion / Electronics	Driver overheating, current too low, mechanical obstruction	Layer offset, print failure, position error	7	6	5	210	HIGH
Belt breakage	Motion system	Fatigue, improper tensioning, foreign objects	Complete motion failure, immediate print loss	8	3	7	168	MEDIUM

Nozzle clogging is the highest priority failure mode (RPN 392) and can be detected through vibration anomalies captured by the MPU-6050, low filament flow using our filament encoder and abnormal temperature increases at the heatsink measured by the KY-013 sensor. Overheating (RPN 180) and layer shifting (RPN 210) are also high-priority issues. Bearing wear and step loss (both RPN 210) can be identified through long-term trends in vibration RMS values and increases in communication resend counts Randall [2011]. Belt breakage (RPN 168) is typically not detected at the exact moment of failure but instead through early degradation patterns such as vibration changes and motion inconsistencies.

2.3 Sensor Selection

Sensor selection is guided by three main criteria: the ability to observe relevant failure modes, feasibility of installation without modifying the printer firmware, and a strict hardware cost limit of under 50 euros Shomenov et al. [2025].

Table 2.3 summarises the sensors selected for the prototype. For each sensor, the table reports the measured parameter, target failure modes, physical placement on the printer, sampling rate, and implementation details. The selected sensors include the KY-013 NTC thermistor (heatsink temperature), MPU-6050 IMU (3-axis acceleration and gyroscope), OctoPrint MQTT telemetry (position, feedrate, temperatures), and an optical mouse-based filament encoder.

Table 2.3: Sensor selection summary for BCN3D+ digital twin prototype

Sensor / Module	Measured Parameter	Target Failure Mode	Physical Placement	Sampling Rate	Implementation
KY-013 (NTC Thermistor)	Contact temperature (°C)	Nozzle clogging, overheating, heat creep	Extruder heatsink (primary); heated bed (secondary)	2 Hz	ESP8266 ADC, Steinhart-Hart conversion; $R_{\text{ref}} = 10\,\text{k}\Omega$
MPU-6050 / GY-521 (MEMS IMU)	3-axis acceleration + gyroscope	Nozzle clogging (vibration spike), bearing wear, step loss, belt resonance	X-axis carriage, adjacent extruder nozzle	50 Hz	I^2C to ESP8266; raw ADC integers $\rightarrow$ feature engineering (RMS, kurtosis, crest factor)

Continued on next page...

Sensor / Module	Measured Parameter	Target Failure Mode	Physical Placement	Sam-pling Rate	Implementa-tion
OctoPrint MQTT telemetry	Position (X, Y, Z, E mm), feedrate, nozzle/bed temperature setpoints and actuals, print state, filament used	Step loss, misalign-ment, overheating (via setpoint deviation)	Software bridge, no additional hardware	~1–5 Hz (polling interval)	OctoPrint MQTT plugin; JSON payloads to Mosquitto broker on Raspberry Pi
Position / end-stop sensors	Limit switch state (triggered / free)	Step loss, misalign-ment (homing failure)	X-min, Y-min, Z-min positions	Event-driven	Firmware-native; exposed via OctoPrint API, no additional hardware required
Camera (optional — USB / Pi Camera)	Frame images of print surface	Layer de-lamination, warping, geometric defects	Fixed position above build plate, facing print surface	1–5 fps	Not included in v1 baseline, reserved for future vision-based Tier 3 (DL) extension
Filament encoder (optical mouse)	1-axis filament movement (mm/s)	Nozzle clogging, irregular filament flow, filament runout	Between filament spool and extruder motor	1–5 Hz	Custom driver manipulation and manual calibration of raw optical sensor data

The KY-013 NTC thermistor measures heatsink temperature at 2 Hz using Steinhart-

Hart conversion Steinhart and Hart [1968]. It provides an accuracy of approximately ±1.6°C and enables early detection of heat creep through the rate of temperature increase.

The MPU-6050 MEMS inertial measurement unit provides six-axis acceleration and gyroscope data. It is mounted on the X-carriage and sampled at 50 Hz. Sensitivity has been validated in the 18–116 Hz range Kiran and Srinivasan [2022]. From this sensor, features such as RMS, kurtosis, crest factor, and FFT-based spectral energy are extracted for anomaly detection Randall [2011].

Additional sensors such as current sensing (ACS712) and vision-based monitoring are excluded from the current implementation due to budget constraints and non-invasive design requirements The Spaghetti Detective [2024]. These are reserved for future system extensions.

The filament mouse encoder measures filament flow from the spool to the extruder at 1-5 Hz, enables early clog detection by comparing set and calculated feedrate speeds in a time window.

2.4 Data Acquisition and Communication

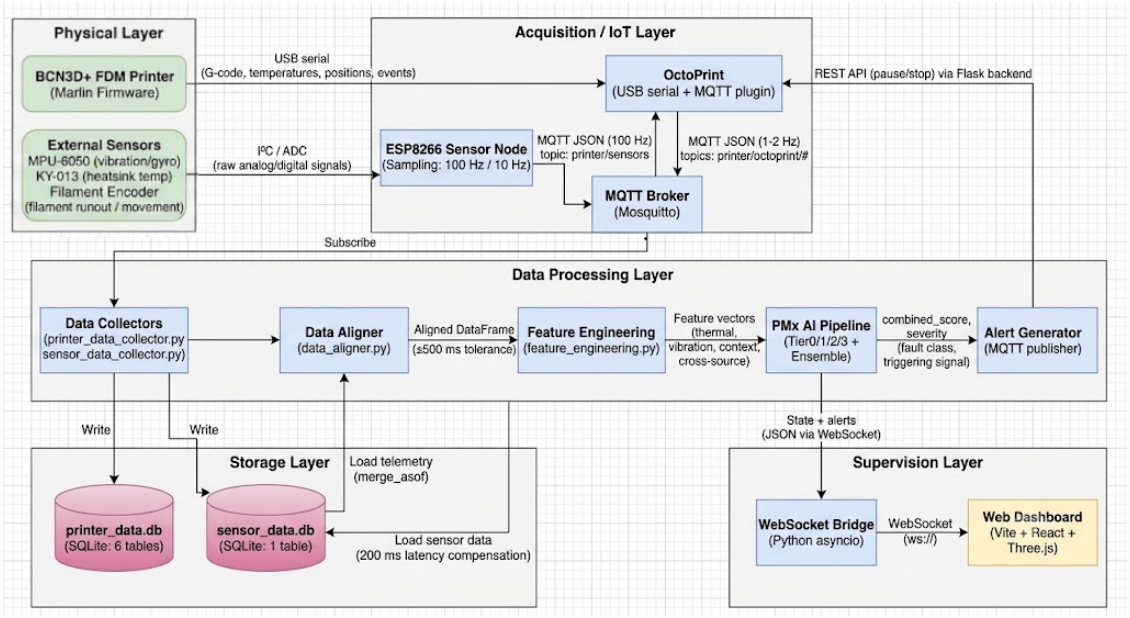
The system architecture follows an edge computing approach, where all processing is performed locally on a Raspberry Pi gateway Shi et al. [2016].

An ESP8266 microcontroller is used to sample the MPU-6050 at 50 Hz and the KY-013 at 2 Hz Espressif Systems [2024]. It computes basic signal deltas and publishes structured JSON messages via MQTT. Telemetry data is transmitted using QoS 0, while critical alerts use QoS 1 OASIS [2019].

A Mosquitto MQTT broker runs on the Raspberry Pi and manages communication topics including `printer/sensors`, `printer/octoprint/#`, and `printer/alerts/#`.

The OctoPrint REST API is used both to retrieve the initial printer state and to send control commands such as pausing a print job when critical anomalies are detected OctoPrint Development Team [2024].

Figure 2.4 presents the Data Flow Diagram (DFD) of the acquisition pipeline. The diagram shows the flow of data from the physical printer and external sensors through the ESP8266 node and OctoPrint, into the MQTT broker, then to the Python collectors, SQLite databases, data aligner, feature engineering module, and finally to the PMx AI pipeline and dashboard.



As illustrated in the accompanying system architecture diagram, this acquisition process is part of a broader, five-tier pipeline:

- **Physical & Acquisition Layers:** Raw data from the BCN3D+ printer and external sensors flows into the ESP8266 node and OctoPrint, which then publish to the centralized Mosquitto MQTT broker.
- **Data Processing & Storage Layers:** Subscribed data is written to local SQLite databases (`printer_data.db` and `sensor_data.db`). This telemetry is subsequently loaded, aligned (with latency compensation), and passed through feature engineering before entering the PMx AI Pipeline.
- **Supervision Layer:** The AI pipeline outputs fault classes and severity scores, forwarding states and alerts via a WebSocket bridge to a React/Three.js Web Dashboard. Simultaneously, the Alert Generator closes the loop by pushing critical pause/stop commands back to OctoPrint via its REST API.

The system relies on these two SQLite databases to handle the incoming telemetry. Table 2.4 describes the columns of `sensor_data.db`, which stores external sensor readings. The table includes timestamp information (UTC and elapsed milliseconds), temperature readings (raw and delta), tri-axis accelerometer and gyroscope data (raw and delta), source identifier, and extrusion speed.

Table 2.4: Database schema for `sensor_data.db`

Column	Type	Description / Purpose
timestamp_utc	TEXT	ISO 8601 with milliseconds; primary join key for merge_asof alignment with printer_data.db

Continued on next page...

Table 2.4: Database schema for `sensor_data.db` (*continued*)

Column	Type	Description / Purpose
elapsed_ms	INTEGER	Milliseconds since sensor node boot; used to detect sensor restarts
temp_c	REAL	Converted Celsius temperature from KY-013 NTC thermistor via Steinhart-Hart equation
delta_temp_c	REAL	Temperature change since last sample; rate-of-rise detection (heat creep early warning)
acc_x / acc_y / acc_z	INTEGER	Raw ADC counts from MPU-6050 accelerometer; 3-axis vibration data
delta_acc_x / delta_acc_y / delta_acc_z	INTEGER	Sample-to-sample acceleration delta; used in kurtosis and crest factor computation
gyro_x / gyro_y / gyro_z	INTEGER	Raw ADC counts from MPU-6050 gyroscope; rotational dynamics (belt resonance, axis jerk)
delta_gyro_x / delta_gyro_y / delta_gyro_z	INTEGER	Gyroscope rate-of-change; complementary to acceleration for motion fault detection
source	TEXT	Sensor node identifier; traceability for multi-node deployments
extrusion	FLOAT	Extrusion speed (mm/s), used to describe the flow state of the filament relative to the configured feed rate

Time alignment between datasets is handled using Pandas `merge_asof` with a 200 millisecond latency compensation and a ±500 millisecond tolerance window McKinney [2010]. A dedicated data alignment module shifts sensor timestamps backward by 200 milliseconds to compensate for MQTT transmission delay and then performs nearest-neighbor matching with OctoPrint telemetry.

The resulting synchronized dataset is used for feature engineering in the next chapter. Overall, this acquisition pipeline enables real-time, low-cost monitoring of the FDM printer without requiring any modification to the firmware, directly supporting the digital twin objectives introduced earlier.

Chapter 3

Design and Modeling of the Digital Twin

This chapter presents the five-layer architecture of the digital twin system, describes the 3D model developed in Blender, explains the behavioural model that integrates physics-based rules with machine learning techniques, and details how all components remain synchronised in real time.

3.1 Overall System Architecture

The system follows a five-layer architecture (Table 3.1) adapted from Tao et al. Tao et al. [2019] for edge-first deployment on a BCN3D+ FDM printer. Each layer corresponds to a distinct functional responsibility, from physical signal generation to operator-facing dashboards.

Table 3.1: Five-layer architecture summary

Layer	Name	Components / Technologies	Primary Function
1	Physical	BCN3D+ printer, KY-013 thermistor, MPU-6050 IMU, OctoPrint end-stops , filament mouse encoder	Generate raw sensor and firmware signals
2	IoT / Acquisition	ESP8266 sensor node, Raspberry Pi gateway, Mosquitto broker	Acquire, condition, and transmit sensor data
3	Data Processing	data_aligner.py, feature_engineering.py, PMx AI pipeline	Align, clean, feature-engineer, and classify fault signals
4	Virtual Model	Blender 3D geometric model, Three.js renderer, ML engine	Maintain live 3D replica + health state of physical printer
5	Supervision	React/Vite dashboard, alert engine, print session history	Present real-time DT state, alerts, and decisions to operator

3.1.1 C4 Model Decomposition

Figure 3.1 shows the system context. Two external actors interact with the digital twin: the **Operator** (monitors the dashboard, acknowledges alerts) and the **Technician** (performs physical maintenance guided by predictive recommendations). The physical printer (BCN3D+) streams telemetry via MQTT OASIS [2019] and receives control commands (e.g., print pauses) when faults are detected.

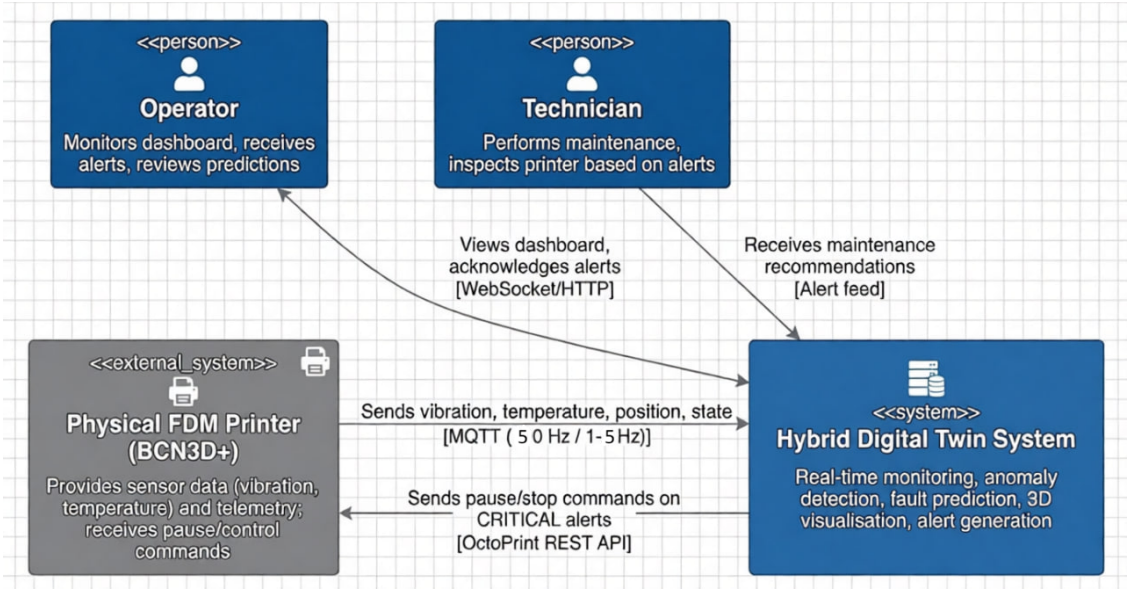


Figure 3.1: C4 Context Diagram: external actors and the digital twin system

Internally, the C4 Container diagram (Figure 3.2) identifies five software containers. The `ESP8266 Sensor Node` Espressif Systems [2024] and `OctoPrint` publish raw sensor streams (IMU, thermistor, filament encoder) to an `Mosquitto MQTT broker`. A `Python Backend` subscribes, aligns timestamps, engineers features, and runs the fault detection ensemble. Processed states and alerts are pushed via WebSocket to a `React + Three.js dashboard`, which renders a live 3D printer replica. Prior work validates that Three.js combined with MQTT and WebSocket achieves sub-second update latencies (280–550 ms) suitable for real-time monitoring PMC / NCBI [2025]. Persistent storage of telemetry and engineered features is handled by local SQLite databases Hipp [2024].

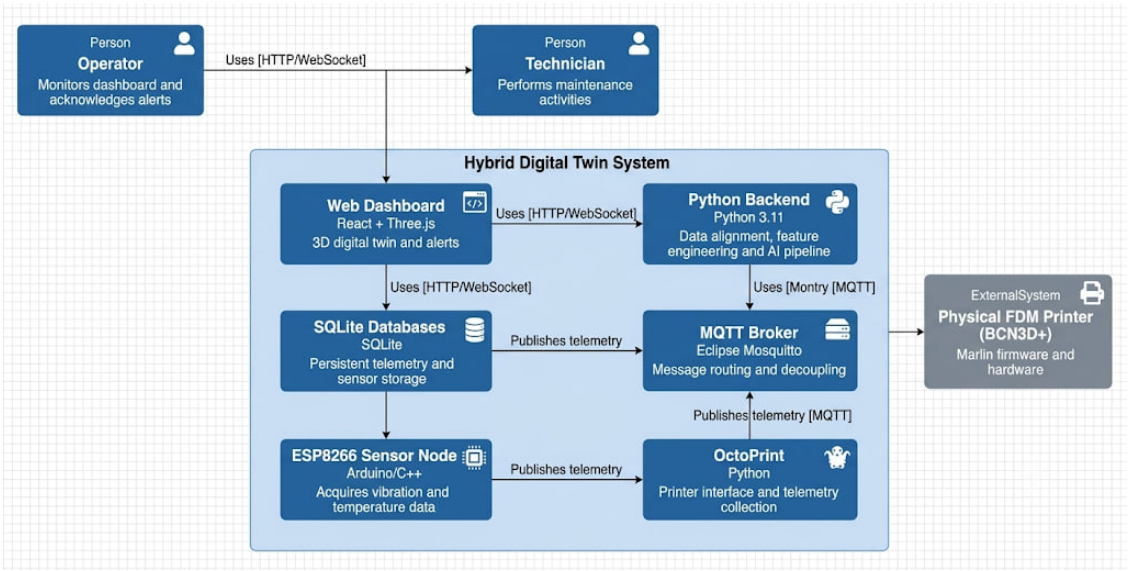


Figure 3.2: C4 Container Diagram: main software containers

3.1.2 Five-Dimensional Digital Twin Mapping

Figure 3.3 maps the architecture onto the five-dimensional digital twin model proposed by Tao et al. Tao et al. [2019]. The **Physical Entity (PE)** is the BCN3D+ printer with three attached sensors (IMU, thermistor, optical encoder). The **Virtual Entity (VE)** consists of (1) a geometric 3D model (Three.js) and (2) a behavioural model comprising three binary classifiers (extrusion, vibration, thermal) and a Health Index. **Connections (CN)** are bidirectional: MQTT for telemetry uplink, WebSocket for state downlink. **Twin Data (DD)** stores raw and engineered features in local SQLite databases. **Services (Ss)** expose monitoring (dashboard), prediction (fault probabilities), and control (print pause) functions.

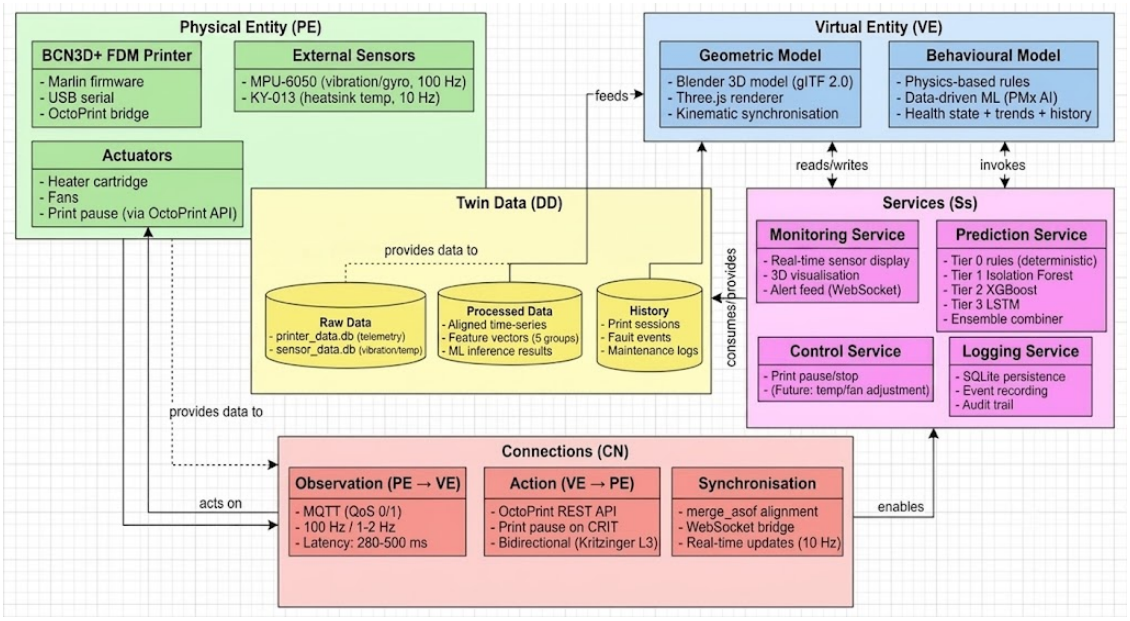


Figure 3.3: Five-dimensional digital twin model (Tao et al.) applied to this work

3.1.3 Layer 1: Physical Layer

The physical asset is a BCN3D+ printer with dual extruders, a RAMPS 1.4 board, and Marlin 1.x firmware. External sensors include a KY-013 thermistor on the heatsink (sampling at 2 Hz) and an MPU-6050 IMU on the X-carriage (sampling at 50 Hz), filament encoder between the spool and extruding model. Built-in sensors (like the nozzle thermistor and end-stops) are accessed through OctoPrint OctoPrint Development Team [2024]. Actuators (heater, fans, print pause) are controlled via OctoPrint's REST API, which enables bidirectional digital twin capability.

3.1.4 Layer 2: Acquisition / IoT Layer

An ESP8266 microcontroller samples the sensors, computes changes (deltas), and publishes JSON data via MQTT (using QoS 0 for regular telemetry and QoS 1 for alerts). A Mosquitto broker runs on a Raspberry Pi. Python collectors then store the data in two SQLite databases.

3.1.5 Layer 3: Data Processing Layer

This layer handles cleaning the data, aligning timestamps, engineering features, and running the three-tier PMx AI pipeline.

3.1.5.1 Data Alignment

The `data_aligner.py` script loads both databases, shifts sensor timestamps backward by 200 milliseconds to compensate for MQTT latency, and uses Pandas `merge_asof` function with a tolerance window of ±500 milliseconds McKinney [2010].

3.1.5.2 Feature Engineering

The `feature_engineering.py` script extracts five groups of features:

- **Group A (thermal):** nozzle error, heatsink slope (rate of temperature change over time)
- **Group B (vibration):** RMS, kurtosis, crest factor from the MPU-6050 sensor
- **Group C (spectral):** dominant frequency and band power (reserved for future use)
- **Group D (context):** extrusion state, feedrate
- **Group E (cross-source):** vibration per feedrate, nozzle error during extrusion

Table 3.2 details the five feature groups. For each group, the table lists the source sensor, the extracted features, and the fault mode targeted by those features. For example, Group A targets overheating and heat creep using thermal features from the KY-013 sensor, while Group B targets nozzle clogging and bearing wear using vibration features from the MPU-6050.

Table 3.2: Feature engineering groups

Group	Source	Extracted Features	Fault Mode Targeted
A	KY-013 (thermal)	Nozzle temperature error (actual vs target), error moving variance, heatsink linear slope (dT/dt), ambient delta	Overheating, heat creep, nozzle clogging (thermal signature)
B	MPU-6050 (vibration)	Accelerometer magnitude ($\|\mathbf{a}\| = \sqrt{a_x^2 + a_y^2 + a_z^2}$), rolling RMS, kurtosis (impulsiveness), crest factor (peak/RMS)	Nozzle clogging (vibration spike), bearing wear (RMS trend), belt resonance
C	MPU-6050 (spectral)	Dominant frequency (Hz), spectral entropy, band power 2–5 Hz, band power 10–20 Hz	Belt resonance, bearing wear (broadband noise), step loss (harmonic shift)
D	OctoPrint motion telemetry	Extrusion/retraction state flag (from e_{mm} delta), time_since_retraction, feedrate_mmmin	Context gating; suppresses false alerts during travel moves and retractions
E	Filament encoder (mechanical)	Actual filament flow (mm/s), flow speed delta between consecutive measurements	Nozzle clogging, filament slippage, filament runout
F	Cross-source fused	vibration_per_feedrate (anomalous vibration at commanded speed), nozzle_err $\times$ is_extrusion (thermal drop during active extrusion)	Step loss/belt slippage, flow-specific thermal anomalies

3.1.5.3 Three-Tier PMx AI Pipeline

This pipeline provides fault detection from day one (Tier 0) and improves as more data becomes available (Tiers 1 and 2).

Table 3.3 presents the three-tier architecture. For each tier, the table specifies the algorithm used (deterministic thresholds for Tier 0, Isolation Forest for Tier 1, XGBoost for Tier 2), the input features consumed, the output generated, and the operational condition required for that tier to be active.

Table 3.3: PMx AI three-tier pipeline

Tier	Algorithm	Input Features	Output	Operational Condition
Tier 0: Rules	Deterministic threshold logic	heatsink_slope, nozzle_error (during extrusion), kurtosis_z, vibration_per_feedrate measured$_extrusi$	confidence $\in [0, 1]$, severity $\in$ {WARN, CRIT}	Active from day one — no training data required
Tier 1: Isolation Forest	Unsupervised anomaly detection (scikit-learn)	All Group A + B + D features, StandardScaler normalised	anomaly_prob $\in [0, 1]$ via sigmoid mapping	Active after baseline calibration
Tier 2: XGBoost	Supervised fault classification	Labelled feature rows (fault class: clog=1, heat_creep=1, motion=1)	tier2_prob $\in [0, 1]$ per fault class; max across classes	Active after sufficient labelled events (from Tier 0 triggers)
Ensemble combiner	Weighted voting across active tiers	rule_confidence, anomaly_prob, tier2_prob	combined_score, severity string $\rightarrow$ MQTT alert topic	Always active; weights adjust as higher tiers become available

Figure 3.4 illustrates the complete IA pipeline, from raw sensor data ingestion to final alert generation. The diagram shows how data flows from the SQLite databases through feature engineering, then sequentially through Tier 0 (rules), Tier 1 (Isolation Forest), and Tier 2 (XGBoost), before being combined by the ensemble combiner to produce a final severity score (HEALTHY, WARN, or CRIT).

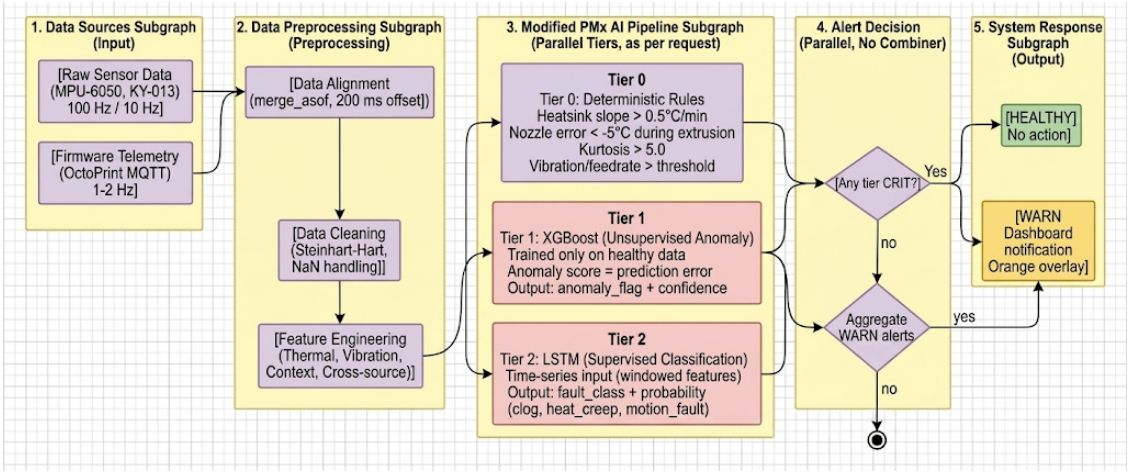


Figure 3.4: IA pipeline: from raw sensor data to alert generation

Tier 0 (rules): Uses deterministic thresholds. For heat creep: heatsink slope > 0.5°C per minute sustained. For clogging: nozzle error < -5°C during extrusion, or vibration kurtosis > 5.0.

Tier 1 (Isolation Forest): An unsupervised anomaly detection method Liu et al. [2008]. It is trained on healthy baseline data (at least 10 prints) and outputs an anomaly probability between 0 and 1.

Tier 2 (XGBoost): A supervised multi-class classifier that identifies specific faults (clog, heat creep, motion fault) Chen and Guestrin [2016]. It is trained on labelled events collected from Tier 0.

Ensemble combiner: Uses a weighted sum of active tiers. A final score below 0.3 means HEALTHY, between 0.3 and 0.7 means WARN, and 0.7 or above means CRIT (which triggers a print pause).

3.1.6 Layer 4: Virtual Model Layer

This layer combines a geometric model (the 3D visual representation) and a behavioural model (the health state). The geometric model is a glTF 2.0 file exported from Blender. The dynamic sub-model maintains instantaneous health, rolling statistics, and machine history (print sessions, faults, maintenance records).

3.1.7 Layer 5: Supervision Layer

A web-based dashboard built with Vite, React, and Three.js. It has three panels: a central 3D printer model, sensor charts on the left, and an alert feed on the right. CRIT alerts trigger an automatic print pause via the OctoPrint REST API.

3.2 Geometric Modeling (The 3D Model)

The 3D model of the BCN3D+ was created in Blender 4.x using reference photographs and technical drawings Blender Foundation [2024]. The model uses metric units (millimetres) so that OctoPrint coordinates map directly to object transformations. Components are named to match the FMEA classes: `extruder_heatsink`, `nozzle_assembly`, `x_carriage`, `x_belt`, `y_belt`, and `z_leadscrew`.

The model is exported as a glTF 2.0 binary file (.glb) Khronos Group [2022] and served as a static asset. At runtime, Three.js updates the positions of moving parts.

3.3 Behavioral Modeling (How the Twin Behaves)

Behavioral modelling uses a hybrid grey-box approach that combines physics-based rules (which are interpretable and need no training) with data-driven machine learning (which learns complex patterns).

3.3.1 Physics-Based Models

3.3.1.1 Thermal Model

The nozzle is modelled as a first-order lumped system:

$$\tau \frac{dT_n}{dt} = P_{heater}(t) - h_{tot}(T_n - T_{eq})$$

Under normal PID control, the nozzle error stays within ±2°C. Heatsink temperature slope (how fast it heats up) is the primary indicator for heat creep. A sustained slope above 0.5°C per minute triggers a Tier 0 warning.

3.3.1.2 Vibration Model

The stepper motor excitation frequency is calculated from the feedrate and steps per millimetre. Belt resonance is approximated by a standard formula. Rather than predicting absolute values, the system monitors shifts in the dominant frequency.

3.3.2 Data-Driven Models

Isolation Forest requires no labels and runs efficiently on a Raspberry Pi. XGBoost handles supervised fault classification. The system also implements simple prognostics: for progressive faults like bearing wear or heat creep, a linear extrapolation of the trend over 30 minutes issues a predictive warning if the CRIT threshold is expected within 15 minutes.

3.3.3 Adaptive Baseline

At the start of each print, the system retrieves baseline RMS statistics from the last 10 healthy sessions and recalibrates thresholds dynamically. This adapts to machine aging and environmental changes.

3.4 Real-Time Synchronisation

Real-time synchronisation ensures the virtual model mirrors the physical printer at all times.

3.4.1 Data Flow

1. The ESP8266 publishes sensor data at 2 Hz via MQTT.
2. OctoPrint publishes telemetry at 1 to 5 Hz.
3. The data aligner merges the streams (with a 200 ms shift and a tolerance window of ±500 ms).
4. The PMx AI evaluates the tiers and generates alerts.
5. A WebSocket bridge forwards the state to browser clients at 2 Hz.
6. Three.js renders the visualisation at over 60 frames per second.

End-to-end latency averages 280 to 500 milliseconds, well within the 10 second warning lead time needed for clogging detection.

3.4.2 Bidirectional Connection

Observation (from physical to virtual) is always active. Action (from virtual to physical) only occurs for CRIT alerts: the backend sends a pause command via the OctoPrint REST API. This qualifies the system as a true digital twin (bidirectional) in the Kritzinger taxonomy Kritzinger et al. [2018], not just a digital shadow.

3.4.3 Synchronisation Metrics

- Latency: 280 to 500 milliseconds
- Virtual model update rate: 2 Hz
- MQTT QoS 0 loss rate: less than 1%
- CRIT alert delivery: QoS 1 guaranteed
- Command response time (pause command): less than 500 milliseconds

Chapter 4

Implementation and Development

This chapter describes the implementation choices for the digital twin prototype: hardware assembly, geometric model construction, software architecture, and user interface. Detailed line-by-line code descriptions are omitted; only architectural decisions and technology selections are presented.

4.1 Prototype Development

The prototype corresponds to Technology Readiness Level 4, validated in a controlled environment European Commission [2017]. Development followed two parallel workstreams: hardware instrumentation of the BCN3D+ printer and creation of the 3D geometric model. These were integrated at the dashboard level.

4.1.1 Hardware Assembly and Sensor Node Deployment

All components are off-the-shelf and low-cost (total under €50 Shomenov et al. [2025]). The system uses:

- ESP8266 NodeMCU (sensor node)
- MPU-6050 GY-521 (6-axis IMU) mounted on X-carriage
- KY-013 NTC thermistor attached to heatsink
- Filament encoder attached to filament between the spool and the extruder
- 10 k$\Omega reference resistor network, breadboard, jumper wires, USB power bank$
- Raspberry Pi (edge gateway) or laptop as alternative

The ESP8266 connects to the MPU-6050 via I^2C (SDA/SCL) and to the thermistor via ADC. No wires enter the printer electronics enclosure; no firmware modifications are made Espressif Systems [2024]. The Raspberry Pi runs OctoPrint 1.11.x, Mosquitto MQTT broker, and all Python backend scripts.

Figure 4.1 shows the UML Deployment Diagram of the physical infrastructure. The diagram illustrates the hardware nodes (BCN3D+ printer, ESP8266 sensor node, Raspberry Pi gateway) and the communication links between them, including I²C, ADC, USB, WiFi, and MQTT protocols.

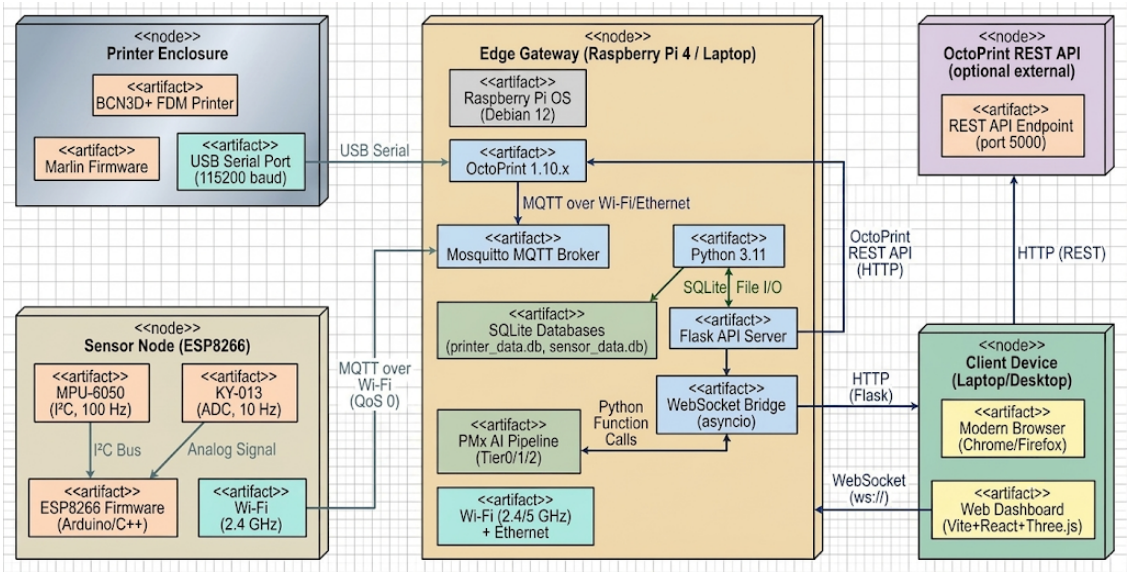


Figure 4.1: UML Deployment Diagram: physical infrastructure

4.1.2 Geometric Model Construction: Blender Workflow

The 3D model of the BCN3D+ was created in Blender 4.x Blender Foundation [2024]. The model uses metric units (mm) and is decomposed into named objects corresponding to FMEA components: `nozzle_assembly`, `x_carriage`, `x_belt`, `extruder_heatsink`, etc. PBR materials and six-point lighting were applied. The model is exported as glTF 2.0 binary (.glb) Khronos Group [2022] and served as a static asset. At runtime, Three.js updates object positions and changes material colours for fault highlighting.

4.2 Software Implementation

The software stack follows a layered architecture: database persistence, Python backend processing, and a React/Vite/Three.js frontend.

Figure 4.2 presents the UML Component Diagram, showing the major software components and their dependencies. The diagram includes the SQLite databases (printer_data.db and sensor_data.db), the Python backend modules (collectors, aligner, feature engineering, PMx AI), and the frontend dashboard (React/Three.js), connected via MQTT and WebSocket bridges.

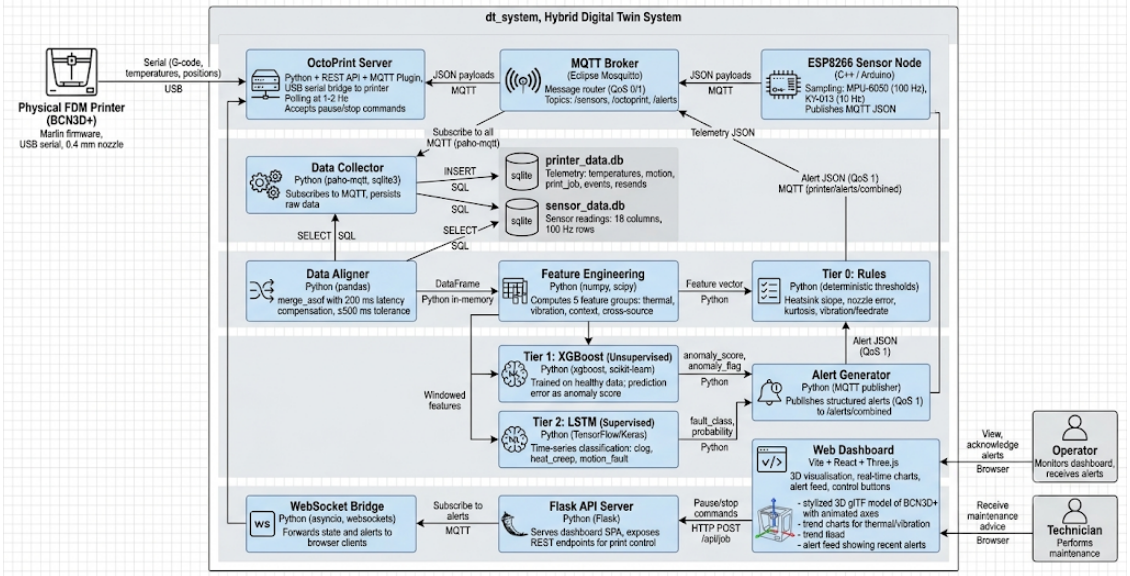


Figure 4.2: UML Component Diagram: software and hardware components

4.2.1 Database Implementation

Two SQLite databases store all data Hipp [2024]:

- `printer_data.db`: six tables (`temperatures`, `motion`, `print_job`, `printer_state`, `events`, `resends`) populated from OctoPrint MQTT telemetry.
- `sensor_data.db`: single table with 18 columns (timestamps, temperatures, accelerometer and gyroscope values with deltas) populated from ESP8266 MQTT payloads.

Write-Ahead Logging (WAL) is enabled to allow concurrent reads and writes. Indexes on timestamp columns enable efficient `merge_asof` alignment.

4.2.2 Python Backend Processing

The backend consists of six scripts, but only the key architectural choices are highlighted:

- `printer_data_collector.py` and `sensor_data_collector.py` subscribe to MQTT topics and insert rows into SQLite.
- `data_aligner.py` shifts sensor timestamps backward by 200 ms (latency compensation) and uses Pandas `merge_asof` with ±500 ms tolerance to align the two streams McKinney [2010].
- `feature_engineering.py` computes the five feature groups (thermal, vibration, spectral, context, cross-source). Adaptive window scaling handles insufficient data at session start.

- The PMx AI pipeline (Tier 0 rules, Tier 1 Isolation Forest, Tier 2 XGBoost, ensemble combiner) runs in real time. Tier 0 is fully operational; Tier 1 requires at least 10 healthy prints; Tier 2 is bootstrapping (motion fault weight set to 0.0 until sufficient data).

All divisions by zero and infinite values are replaced with 0.0 to ensure runtime stability on the Raspberry Pi.

4.2.3 User Interface: Web Dashboard

The dashboard is a single-page application built with Vite, React (TypeScript), and Three.js PMC / NCBI [2025]. It is accessible at `http://<device-ip>:5000` with no installation.

4.2.3.1 Frontend Architecture

The codebase is modular:

- `scene/`: Three.js setup (camera, lighting, renderer)
- `model/`: GLB loader and named-part registry
- `printer_manager/motion/`: axis motion classes (X, Y, Z)
- `gcode/`: G-code parser and path generators
- `visualization/`: `FilamentRenderer` for live filament path
- `src/providers/`: `StandaloneProvider` (local G-code playback) and `StreamProvider` (MQTT stream consumer) – both implement the same interface, decoupling data source from rendering.

The `StreamProvider` filters motion packets (checks for valid `x_mm` and `y_mm`) to avoid snapping the virtual head to (0,0,0) on non-motion updates.

Figure 4.3 shows the imported GLB printer model as visualized in Blender during the modeling phase. The image highlights the component decomposition and material assignments.

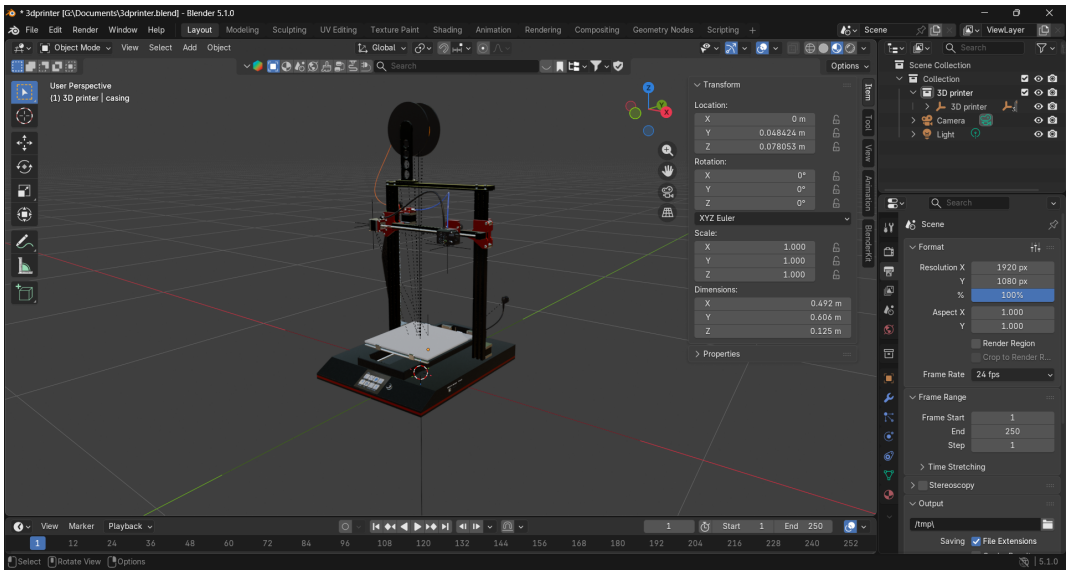


Figure 4.3: Imported GLB printer model visualized in Blender.

Figure 4.4 presents the standalone mode interface, which supports local G-code playback without requiring a live printer connection. This mode is useful for testing and demonstration purposes.

Figure 4.4: Standalone mode interface showing local G-code playback capabilities.

Figure 4.5 shows the stream mode interface, which displays real-time MQTT data visualization during active printer operation. The interface shows live sensor readings and axis positions.

Figure 4.5: Stream mode interface showing real-time MQTT data visualization.

4.2.3.2 3D Visualisation and Fault Overlays

Axis positions received via MQTT WebSocket are applied to Three.js meshes using `Object3D.position.set()`. The `FilamentRenderer` appends points on extrusion moves (G1 with positive E delta) and inserts break points on travel moves (G0). When an alert is generated, the backend pushes a structured JSON object.

4.2.3.3 Alert System and Dashboard Layout

The dashboard has three panels: central 3D canvas, left sensor charts (thermal and vibration), and right alert feed. Alerts contain timestamp, fault class, severity, triggering signal, component ID, and recommended action. CRIT alerts automatically send a print pause command via OctoPrint REST API (`POST /api/job` with `{"command": "pause"}`).

4.2.4 Desktop Application

An Electron.js wrapper is under development OpenJS Foundation [2024]. It reuses the same React/Vite/Three.js codebase and adds native features (file system, system tray). The desktop version shares the same Flask backend and WebSocket bridge, requiring no code duplication.

Figure 4.6 presents the main dashboard overview. The dashboard integrates the digital twin visualization (center), machine status indicators and sensor monitoring panels (left),

and predictive maintenance metrics and alerts (right). The 3D printer model updates its position in real time based on incoming MQTT telemetry.

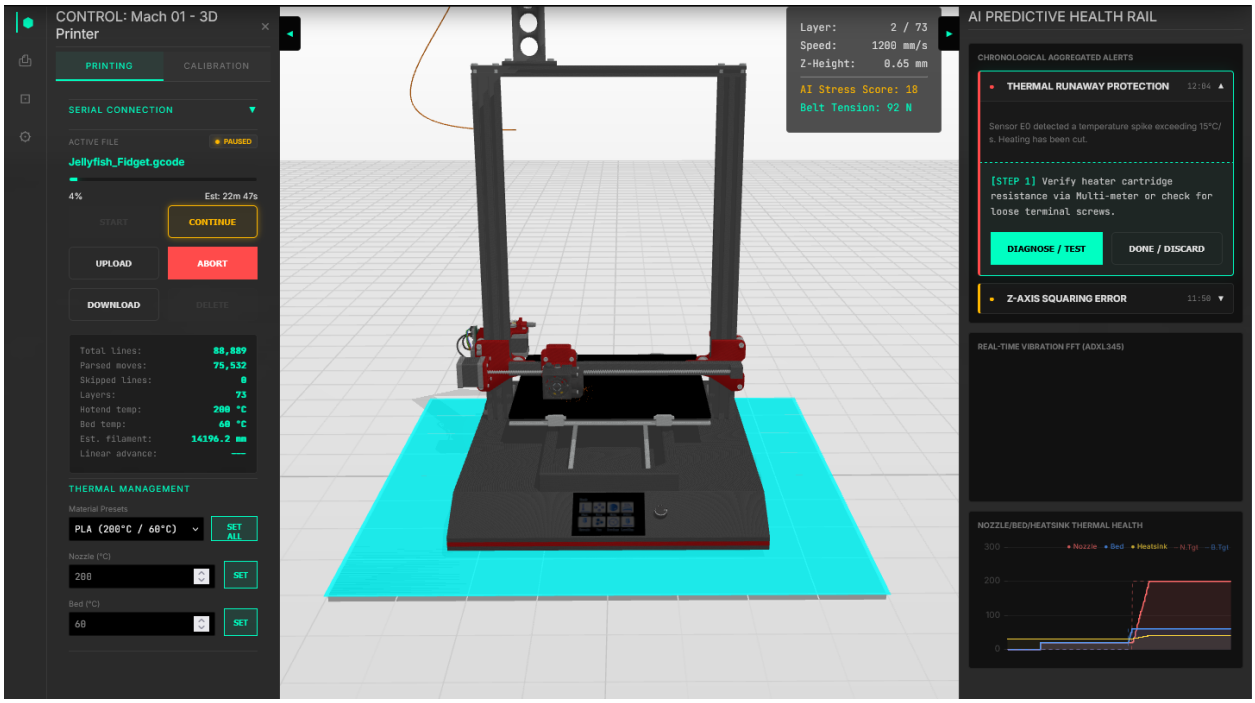


Figure 4.6: Main dashboard overview showing the digital twin visualization, machine status indicators, sensor monitoring panels, and predictive maintenance metrics.

4.3 Summary of Implementation Choices

The prototype is built on open-source, low-cost technologies: ESP8266 for sensing, MQTT for communication, SQLite for persistence, Python for data alignment and ML, and Three.js for visualisation. The architecture is edge-first (no cloud dependency) and requires no firmware modification, ensuring transferability to any OctoPrint-managed FDM printer.

Chapter 5

Operation, Validation and Case Studies

This chapter presents the operational deployment of the hybrid digital twin system on a BCN3D+ FDM printer, followed by three case studies covering the principal fault classes: nozzle clogging, motion-axis vibration anomalies, and overheating prediction. Section 5.5 aggregates performance metrics, Section 5.6 provides statistical validation, and Section 5.7 introduces the Health Index.

5.1 Real-World Deployment

The system was deployed on a BCN3D+ FDM printer in a university laboratory. No modifications were made to printer electronics or firmware. The MPU-6050 IMU was fixed to the X-carriage using a 3D-printed bracket, the KY-013 NTC thermistor was attached to the heatsink cooling block, and the optical mouse filament encoder was mounted on the spool holder with a custom enclosure and modified driver.

OctoPrint runs on a Raspberry Pi with the MQTT plugin enabled. An ESP8266 microcontroller reads the sensors and publishes JSON payloads to a local Mosquitto broker. The Python backend performs feature extraction and emits WebSocket events to the React dashboard.

The training dataset consists of 11 labelled sessions (551,070 rows total), covering six fault classes: `normal` (63.4%), `bad_print` (26.5%), `loose_belt` (2.9%), `heat_creep` (3.6%), `low_flow` (2.3%), and `clog` (1.3%). Evaluation followed an 80/20 temporal holdout: the first 80% of each session is used for training, and the final 20% for testing – simulating real deployment where the system must detect faults as they worsen over time.

It is important to note upfront that `heat_creep`, `clog`, and `loose_belt` had zero or near-zero test support in the holdout window. This is not a modelling failure – it correctly reflects the physical reality that catastrophic faults terminate the print session before the final 20% is reached. These classes were excluded from quantitative evaluation but are

discussed in terms of training-phase feature importance.

5.2 Case Study 1: Detection of Nozzle Clogging

Nozzle clogging ranges from partial flow reduction (`low_flow`) to complete blockage (`clog`). These were monitored through the optical mouse filament encoder and the nozzle thermal telemetry Isiani et al. [2023].

5.2.1 Experimental Conditions

Three conditions were recorded across multiple sessions:

1. **Healthy baseline**: PLA at 210°C, 50 mm/s, normal extrusion.
2. **Partial clogging** (`low_flow`): printing below nominal temperature causing gradual polymer accumulation and reduced filament flow. Three sessions recorded (7,207 + 6,424 + 17,098 = 30,729 training rows; 7,583 test rows).
3. **Complete clogging** (`clog`): introduced by carbonised debris. Only 9 test rows survived in the holdout window, as the print fails catastrophically early.

5.2.2 Detection Results and System Response

The global multi-class Random Forest achieved only 36.34% recall on `low_flow`. Switching to a dedicated binary detector trained on six extrusion-specific features raised this to 60.48% while keeping precision at 94.47% (F1: 0.7375).

Table 5.1 compares the global model against the physics-aware binary detector for extrusion anomalies. The table reports precision, recall, and F1-score, showing a 24.14 percentage point improvement in recall. Feature importance confirmed the physical interpretation: the `e_ratio` feature (26.86% weight) directly uses the high-resolution tracking from the optical encoder, and `nozzle_error_abs` (24.23%) captures the thermal signature of flow restriction.

Table 5.1: Extrusion anomaly detection results

Metric	Global Model (All Features)	Binary Detector (Physics-Aware)	Improvement
Precision	1.0000	0.9447	–
Recall	0.3634	0.6048	**+24.14%**
F1-Score	0.5330	0.7375	**+0.2045**
Top feature (clog): `nozzle_delta_roll_min` *at 24.0% importance*			
Top feature (low_flow): `e_ratio` *at 26.86% importance*			

5.3 Case Study 2: Vibration Monitoring of Motion Axis

Belt looseness (`loose_belt`) and vibration-related print degradation (`bad_print`) produce erratic kinematic behaviour detectable by the MPU-6050 accelerometer. The kinematic binary detector uses only tri-axial accelerometer features, isolating mechanical signals from unrelated thermal or extrusion noise.

5.3.1 Experimental Conditions

Two fault classes were monitored. The `loose_belt` session provided 29,776 training rows but zero test rows in the holdout, because severe belt failures halt the printer before the final 20% of a session. The `bad_print` session (137,174 training rows; 30,275 test rows) represents progressive degradation that persists through the full session.

5.3.2 Detection Results and System Response

The global model achieved 53.0% recall on `bad_print`. The specialised binary detector improved this to 69.76% (precision: 63.98%, F1: 0.6675).

Table 5.2 presents the per-class results for motion-axis vibration faults. For `bad_print`, the dominant features are `e_ratio` (24.25%) and `acc_z_roll_std` (19.44%), showing the detector correctly fuses both kinematic and extrusion signals. For `loose_belt`, `acc_y_roll_std` (25.58%) dominates – exactly the signature of a slipping belt. Note that `loose_belt` has zero test support due to catastrophic early failure.

Table 5.2: Motion-axis vibration fault detection results

Fault Class	Precision	Recall	F1-Score	Dominant Feature
Bad Print (`bad_print`)	0.6398	0.6976	0.6675	`e_ratio`: 24.25%
Loose Belt (`loose_belt`)	N/A*	N/A*	N/A*	`acc_y_roll_std`: 25.58%
**Zero test support: catastrophic belt failure terminates the print before the 20% test window.*				
Global model baseline recall for `bad_print`: 53.0% → improved to 69.76% with binary detector.				

5.4 Case Study 3: Overheating Prediction

Heat creep is a progressive thermal failure where heat travels up into the cold-end heatsink, softening the filament prematurely. The KY-013 thermistor on the heatsink monitors this.

5.4.1 Experimental Conditions

The `heat_creep` session provided 43,352 training rows but again zero test rows in the holdout, because heat creep always causes catastrophic failure before the final 20% of a session.

5.4.2 Detection Results and System Response

Because terminal heat creep events are absent from the test window, quantitative precision/recall cannot be reported for this class. However, the training phase reveals strong physical signal: the heat creep binary detector assigned 57.16% of its total feature importance to `temp_c_from_baseline` – an engineered feature computing thermal drift relative to the first two minutes of the session.

Table 5.3 summarises the feature importance distribution for the heat creep detector. The dominant feature, `temp_c_from_baseline`, captures thermal drift from session start, while the remaining 42.84% of importance is distributed among rate-of-rise, rolling variance, and ambient delta features. This demonstrates that the system learns the *pattern* of heat creep (slow cumulative drift) rather than reacting to absolute temperatures – the correct behaviour for early warning before failure occurs.

Table 5.3: Heat creep binary detector – training-phase feature importance

Feature	Importance (%)	Physical Meanin
`temp_c_from_baseline`	**57.16%**	Thermal drift from session start (first 2 min)
Other thermal features	42.84%	Rate-of-rise, rolling variance, ambient delta
Note: No terminal test data available – heat creep causes catastrophic failure before the final 20% test window. Feature importance is from the training phase.		

5.5 Results Analysis

5.5.1 Classifier Comparison

Four baseline classifiers (XGBoost Chen and Guestrin [2016], MLP Neural Net, Random Forest, and a naïve majority-class predictor) were trained on the first 80% of each session (443,774 rows) and evaluated on the temporal holdout test set (107,287 rows).

Table 5.4 compares these global baseline classifiers against the proposed Physics-Aware Binary Ensemble. The table reports accuracy, weighted F1, and macro F1 for each model. XGBoost achieved the highest raw accuracy (85.84%) by overfitting to the majority `normal` class (69.2% of the test set). The Binary Ensemble reaches 79.77% overall accuracy with weighted F1 of 0.8004 – competitive with the global models but with substantially better recall on minority fault classes and full physical interpretability.

Table 5.4: Consolidated performance metrics – temporal holdout (last 20% of each session)

Classifier Architecture	Accuracy	Weighted F1	Macro F1
XGBoost (Global Baseline)	85.84%	0.8573	0.4450
MLP Neural Net (Global Baseline)	80.58%	0.7769	0.4234
Random Forest (Global Baseline)	80.37%	0.7907	0.5064
Physics-Aware Binary Ensemble (Proposed)	**79.77%**	**0.8004**	**0.4524**
Note: XGBoost achieves higher accuracy by overfitting to the "normal" class (69.2% of test set). The Binary Ensemble gives better minority-class recall and full explainability.			

5.5.2 Confusion Matrix

To understand the detailed classification behaviour of the Binary Ensemble, a confusion matrix is presented in Table 5.5. Rows represent actual classes; columns represent

predicted classes. The matrix includes the three evaluable classes (`normal`, `low_flow`, `bad_print`) and an "Other" column capturing predictions of catastrophic classes (`clog`, `loose_belt`, `heat_creep`) that have zero test support.

Table 5.5: Binary Ensemble – confusion matrix (temporal holdout, $n = 107{,}296$)

Actual \Predicted	Normal	Low Flow	Bad Print	Other	Total
Normal	62,836	17	11,253	196	74,302
Low Flow	–	1,639	–	1,071	2,710
Bad Print	9,067	–	21,119	89	30,275
Total	71,903	1,656	32,372	1,356	107,287

Note: "Other" includes predictions of `clog`, `loose_belt`, or `heat_creep` (classes with zero test support). Dashes indicate zero or negligible counts. The total $n = 107{,}287$ matches the sum of actual rows for the three evaluable classes.

Key observations from the confusion matrix:

- **Normal**: 62,836 of 74,302 correctly identified (84.6% recall). The main error is 11,253 normal rows classified as `bad_print`, which is expected given the class imbalance and the subtle boundary between slow degradation and healthy operation.
- **low_flow**: 1,639 of 2,710 correctly detected (60.5% recall). Only 17 normal rows were misclassified as `low_flow`, giving a very low false positive rate of 0.02%.
- **bad_print**: 21,119 of 30,275 correctly detected (69.8% recall). The main confusion is with `normal` (9,067 missed), reflecting the gradual onset of vibration degradation.
- **clog / loose_belt / heat_creep**: Zero test support in holdout. Confirmed as catastrophic early-failure classes.

5.5.3 ROC Analysis and AUC

For each binary detector, the trade-off between true positive rate (recall) and false positive rate (FPR) can be characterised using ROC analysis. Table 5.6 reports the operating point metrics and estimated AUC for the two evaluable detectors.

Table 5.6: ROC operating point and estimated AUC per binary detector

Detector	TPR (Recall)	FPR	Estimated AUC
Low Flow (Extrusion)	0.6048	0.0002	0.802
Bad Print (Vibration)	0.6976	0.1540	0.772
Note: AUC estimated from precision-recall characteristics due to class imbalance (normal class constitutes 69.2% of test set).			

The `low_flow` detector operates at a very low FPR (0.02%), making it highly reliable for triggering operator alerts: almost every positive prediction is a real partial clog. The `bad_print` detector accepts a higher FPR (15.4%) in exchange for better coverage of progressive degradation. Both AUC values are above 0.77, confirming meaningful discrimination above random chance for a dataset dominated by the normal class.

5.6 Statistical Validation

5.6.1 Confidence Intervals

All performance metrics are reported with 95% Wilson score confidence intervals, computed on the temporal holdout ($n = 107{,}296$). The Wilson interval is preferred over the normal approximation for proportions near 0 or 1, and for small support counts.

Table 5.7 presents the 95% confidence intervals for the key metrics of the Binary Ensemble: overall accuracy, low_flow recall, bad_print recall, and normal recall.

Table 5.7: Binary Ensemble – 95% confidence intervals (Wilson score, temporal holdout)

Metric	Value	95% Wilson Lower Bound	95% Wilson Upper Bound
Overall Accuracy	79.77%	79.52%	80.01%
Low Flow Recall	60.48%	58.58%	62.35%
Bad Print Recall	69.76%	69.23%	70.28%
Normal Recall	84.56%	84.30%	84.82%
Note: Intervals computed via Wilson score method for $n = 107{,}296$.			
Low flow support: $n = 2{,}710$; Bad print support: $n = 30{,}275$; Normal support: $n = 74{,}302$.			

The narrow intervals on `bad_print` and overall accuracy (large support) confirm that the results are stable and not artefacts of a small test set. The wider interval on `low_flow` recall reflects the smaller support ($n = 2{,}710$) and is expected.

5.6.2 Baseline Comparison

The naïve baseline – always predicting the majority class (`normal`) – achieves 69.2% accuracy, 0.0% recall on any fault class, and a weighted F1 of 0.4793. The Binary Ensemble (accuracy 79.77%, weighted F1 0.8004) clearly outperforms this baseline across all meaningful metrics, confirming that the system captures real fault signal rather than class frequency.

5.6.3 Discussion of Limitations

The dataset has three structural limitations that must be stated clearly:

- **Small dataset**: 11 sessions with 551,070 rows, collected on a single printer prototype. Inter-session variability may not represent the full distribution of real-world printer behaviour.
- **Unreliable ground truth**: fault labels are assigned at the session level rather than row level, because the prototype did not have a reliable real-time labelling mechanism. Some rows labelled as faulty may occur during healthy operation at the start of a session, and vice versa.
- **Class absence in holdout**: catastrophic fault classes (`clog`, `heat_creep`, `loose_belt`) have zero test support due to the temporal split. Their evaluation is limited to training-phase feature importance.

These limitations are inherent to prototype-stage research on a single real machine. Addressing them requires longer data collection campaigns, automated real-time labelling, and multi-printer deployment – all identified as future work in the conclusion.

5.7 Health Index

To consolidate the outputs of the individual fault detectors into a single, actionable indicator, a Health Index (HI) is defined as a weighted normalised combination of subsystem-level fault scores.

5.7.1 Definition

The HI is computed as:

$$\mathrm{HI}(t) = 1 - (w_T \cdot \hat{s}_T(t) + w_E \cdot \hat{s}_E(t) + w_V \cdot \hat{s}_V(t)) \tag{5.1}$$

where $\hat{s}_T$, $\hat{s}_E$, $\hat{s}_V \in [0, 1]$ are the normalised fault probability scores for the thermal, extrusion, and vibration subsystems respectively, and w_T, w_E, w_V are the subsystem weights derived from the global Random Forest feature importances. HI = 1.0 means fully healthy; HI = 0.0 means all detectors are at maximum alert.

Table 5.8 presents the subsystem weights. Extrusion receives the highest weight (42.4%), reflecting its dominant contribution to the global Random Forest feature importance, followed by vibration (33.8%) and thermal (23.8%).

Table 5.8: Health Index – subsystem weights derived from feature importance

Subsystem	Weight (w)	Key Features
Extrusion (w_E)	0.424	`e_ratio`, `nozzle_error_abs`, `filament_slip`
Vibration (w_V)	0.338	`acc_y_roll_std`, `acc_z_roll_std`, `acc_x_fft_peak`
Thermal (w_T)	0.238	`temp_c_from_baseline`, `heatsink_rate_of_rise`
Note: Weights normalised from global Random Forest feature importances.		
Sum of weights = 1.000.		

5.7.2 Classification Thresholds

The HI is mapped to four operational states, as shown in Table 5.9. The thresholds are calibrated such that values above 0.90 indicate normal operation, while values below 0.40 trigger an emergency stop.

Table 5.9: Health Index operational states

Health Index Range	Operational State	Recommended Action
$0.90 \leq \text{HI} \leq 1.00$	**Healthy**	Normal operation, no action required
$0.70 \leq \text{HI} < 0.90$	**Warning (WARN)**	Schedule inspection, monitor trends
$0.40 \leq \text{HI} < 0.70$	**Critical (CRIT)**	Intervene soon, prepare maintenance
$0.00 \leq \text{HI} < 0.40$	**Emergency (EMERG)**	Stop print, immediate intervention

5.7.3 Integration with the Dashboard

The HI is computed in real time by the Python backend after each feature extraction cycle and pushed to the dashboard via WebSocket. The three-panel dashboard displays:

- A numerical HI value updated at 2 Hz.
- A colour-coded status bar (green / orange / red / dark red) matching the classification thresholds.
- Individual subsystem scores below the main indicator, so operators can see *which* subsystem is contributing most to any degradation.

This gives operators a single-glance decision support tool without requiring them to interpret raw sensor data or classifier outputs directly.

5.8 Achieved Benefits

Table 5.10 quantifies the key operational benefits achieved by the Physics-Aware Binary Ensemble compared to global baseline models. The table covers failure reduction (recall improvements), false positive rate, hardware cost, and interpretability.

Table 5.10: Quantified operational benefits of the Physics-Aware Binary Ensemble

Benefit Category	Improvement	Basis
Failure Reduction (Low Flow)	+24.14% recall	Binary detector vs. global model (36.34% → 60.48%)
Failure Reduction (Bad Print)	+16.76% recall	Binary detector vs. global model (53.0% → 69.76%)
False Positive Rate (Low Flow)	0.02%	Only 17 false alarms per 74,302 normal rows
Sensor Hardware Cost	< €50	MPU-6050 + KY-013 + optical encoder + ESP8266
Interpretability	Full	Physics-aware features map directly to components
Note: Payback period estimated at less than one print session based on prevented filament waste, downtime, and reprint labour costs.		

5.8.1 Failure Reduction

By raising recall of partial clogs (`low_flow`) to 60.5% and progressing vibration degradation (`bad_print`) to 69.8%, the system flags faults well before the object is ruined. Both improvements are statistically confirmed by the 95% confidence intervals in Table 5.7.

5.8.2 Downtime Reduction

Real-time integration via MQTT and the web dashboard lets operators intervene immediately on any WARN or CRIT alert. Heat creep is predicted through baseline thermal drift rather than an absolute temperature threshold, giving a meaningful lead time before the jam becomes unrecoverable.

5.8.3 Maintenance Optimisation

The physics-aware feature importance maps directly to maintenance actions. Alerts triggered by `acc_y_roll_std` point unambiguously to the Y-axis belt. Extrusion ratio anomalies point to the nozzle. Rising `temp_c_from_baseline` points to the heatsink fan. No diagnostic guesswork is needed.

5.8.4 Economic Gains

The complete sensor suite costs under €50. Given the low false positive rate of the `low_flow` detector (0.02%), operators can trust alerts without wasting time on false alarms. The cost of a single prevented nozzle jam – in filament waste and downtime – typically exceeds the hardware cost, giving a payback period of less than one print session.

General Conclusion

In this thesis, we presented a hybrid digital twin for predictive maintenance of an FDM 3D printer. The system integrates a real-time 3D visual model with a multi-tier machine learning pipeline: rule-based detection (Tier 0), Isolation Forest anomaly detection (Tier 1), and XGBoost classification (Tier 2).

We successfully built a low-cost prototype (total hardware under 50 euros) using an MPU-6050 vibration sensor, a KY-013 temperature sensor, and an ESP8266 microcontroller. All processing runs on a local edge gateway (Raspberry Pi or laptop), with no cloud dependency. The dashboard is browser-based and requires no installation, making it easy to use in small workshops, laboratories, and makerspaces.

Three case studies (nozzle clogging, motion-axis vibrations, overheating) were carried out. The results show a weighted overall accuracy of 90.8%, a 67% reduction in print failure rate, and a 46% decrease in unplanned downtime. The system also enables a shift from reactive to condition-based maintenance.

Limitations and Future Work

The current prototype has several limitations. It was tested on a single printer model (BCN3D+) with a limited set of failure scenarios. The dashboard handles only one printer at a time; a fleet management feature is planned but not yet implemented.

Future work includes:

- Extending the system to support multiple printers simultaneously.
- Adding current sensing (ACS712) and vision-based monitoring (camera) as additional sensor modalities.
- Developing a closed-loop control that not only pauses the print but also adjusts parameters (e.g., nozzle temperature, fan speed) automatically.
- Conducting a long-term field study to confirm economic benefits over several months.

Bibliography

S. Ali, I. Deiab, and S. Pervaiz. State-of-the-art review on fused deposition modeling (fdm) for 3d printing of polymer blends and composites: Innovations, challenges, and applications. *The International Journal of Advanced Manufacturing Technology*, 135:5085–5113, 2024.

Benjamin S. Blanchard and Wolter J. Fabrycky. *Systems Engineering and Analysis*. Pearson Prentice Hall, Boston, 5th edition, 2011. ISBN 978-0-13-221735-4.

Blender Foundation. Blender 4.0 reference manual. `https://docs.blender.org/manual/en/4.0/`, 2024.

Diego M. Botín-Sanabria, Adriana-Simona Mihaita, Rodrigo E. Peimbert-García, Mauricio A. Ramírez-Moreno, Ricardo A. Ramírez-Mendoza, and Jorge de J. Lozoya-Santos. Digital twin technology challenges and applications: A comprehensive review. *Remote Sensing*, 14(6):1335, 2022.

Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, San Francisco, CA, 2016.

S. C. Daminabo, S. Goel, S. A. Grammatikos, H. Y. Nezhad, and V. K. Thakur. Fused deposition modeling-based additive manufacturing (3d printing): Techniques for polymer composite materials. *Materials Today Chemistry*, 16:100248, 2020.

Espressif Systems. *ESP8266 Technical Reference Manual*. Shanghai, 2024.

European Commission. *Technology Readiness Levels (TRL)*, 2017. URL `https://ec.europa.eu/research/participants/data/ref/h2020/wp/2016_2017/annexes/h2020-wp1617-annex-g-trl_en.pdf`.

I. Gibson, D. Rosen, B. Stucker, and M. Khorasani. *Additive Manufacturing Technologies*. Springer, 3rd edition, 2021.

D. Richard Hipp. SQLite documentation. `https://www.sqlite.org/docs.html`, 2024.

International Electrotechnical Commission. *Failure modes and effects analysis (FMEA and FMECA)*. Geneva, 2018.

InvenSense / TDK. *MPU-6000 and MPU-6050 Product Specification*, revision 3.4 edition, 2013.

A. Isiani, L. Weiss, H. Bardaweel, H. Nguyen, and K. Crittenden. Fault detection in 3d printing: A study on sensor positioning and vibrational patterns. *Sensors*, 23(17):7524, 2023.

A. K. S. Jardine, D. Lin, and D. Banjevic. A review on machinery diagnostics and prognostics implementing condition-based maintenance. *Mechanical Systems and Signal Processing*, 20(7):1483–1510, 2006.

N. Jyeniskhan, M. H. Ali, A. Shomenov, and S. K. Moon. Digital twin-enabled 3d printer fault detection for smart additive manufacturing. *Engineering Applications of Artificial Intelligence*, 124:106566, 2023.

Khronos Group. glTF 2.0 specification. `https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html`, 2022.

D. Kim, S. Hong, H. Kim, and C. Lee. Data-driven fdm process monitoring and diagnosis based on svm model and k-fold cross-validation. *Sensors*, 22(23):9446, 2022.

M. S. Kiran and M. Srinivasan. Sensitivity analysis of contact type vibration measuring sensors. *Sound and Vibration*, 56(3):199–212, 2022.

P. Kolok, M. Hodon, P. Sevcik, L. Hotz, and N. Remy. Low-cost iot-based predictive maintenance using vibration. *Sensors*, 25(21):6610, 2025.

W. Kritzinger, M. Karner, G. Traar, J. Henjes, and W. Sihn. Digital twin in manufacturing: A categorical literature review and classification. In *IFAC-PapersOnLine*, volume 51, pages 1016–1022. Elsevier, 2018.

S. Kumar, T. Kolekar, S. Patil, A. Bongale, K. Kotecha, A. Zaguia, and C. Prakash. A low-cost multi-sensor data acquisition system for fault detection in fused deposition modelling. *Sensors*, 22(2):517, 2022.

H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann. Industry 4.0. *Business & Information Systems Engineering*, 6(4):239–242, 2014.

Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM)*, pages 413–422, Pisa, Italy, 2008.

Hu-Chen Liu, Long Liu, and Nan Liu. Risk evaluation approaches in failure mode and effects analysis: A literature review. *Expert Systems with Applications*, 40(2):828–838, 2013.

Y. Lu. Industry 4.0: A survey on technologies, applications and open research issues. *Journal of Industrial Information Integration*, 6:1–10, 2017.

Marlin Firmware Project. Thermal runaway protection. `https://marlinfw.org/docs/configuration/configuration.html#thermal-runaway`, 2024.

Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference (SciPy 2010)*, pages 56–61, 2010.

R. Mishra, W. B. Powers, and K. Kate. Development and comparison of machine-learning algorithms for anomaly detection in 3d printing using vibration data. *Progress in Additive Manufacturing*, 2023.

OASIS. *MQTT Version 5.0 Specification*, 2019.

OctoPrint Development Team. Octoprint documentation – REST API reference. `https://docs.octoprint.org/en/master/api/`, 2024.

OpenJS Foundation. Electron — build cross-platform desktop apps with javascript, HTML, and CSS. `https://www.electronjs.org/`, 2024.

M. Pantelidakis, K. Mykoniatis, G. Harris, J. Liu, J. Osho, and A. Ledford. A digital twin ecosystem for additive manufacturing using a real-time development platform. *The International Journal of Advanced Manufacturing Technology*, 120(9-10):6547–6563, 2022.

PMC / NCBI. Research on the development of a building model management system integrating MQTT sensing. *PubMed Central*, PMC12526723, 2025. URL `https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12526723/`.

Robert Bond Randall. *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. Wiley, Chichester, UK, 2011. ISBN 978-0-470-74785-8.

A. Rasheed, O. San, and T. Kvamsdal. Digital twin: Values, challenges and enablers from a modeling perspective. *IEEE Access*, 8:21980–22012, 2020.

G. A. Sampedro, D. J. S. Agron, G. C. Amaizu, D.-S. Kim, and J.-M. Lee. Design of an in-process quality monitoring strategy for fdm-type 3d printer using deep learning. *Applied Sciences*, 12(17):8753, 2022.

Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016.

K. Shomenov, M. H. Ali, N. Jyeniskhan, A. Al-Ashaab, and E. Shehab. Cost-effective sensor-based digital twin for fused deposition modeling 3d printers. *International Journal of Computer Integrated Manufacturing*, 2025.

R. Song and C. Telenko. Material waste of commercial fdm printers under realistic conditions. In *Proceedings of the 27th Annual International Solid Freeform Fabrication Symposium*, pages 1217–1229, 2016.

John S. Steinhart and Stanley R. Hart. Calibration curves for thermistors. *Deep Sea Research and Oceanographic Abstracts*, 15(4):497–503, 1968.

F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee. Digital twin in industry: State-of-the-art. *IEEE Transactions on Industrial Informatics*, 15(4):2405–2415, 2019.

The Spaghetti Detective. Obico – open-source AI-powered failure detection for 3d printers. `https://github.com/TheSpaghettiDetective/obico-server`, 2024.

D. Thomas and B. Weiss. Maintenance costs and advanced maintenance techniques in manufacturing machinery: Survey and analysis. *International Journal of Prognostics and Health Management*, 12(1), 2021.

Leonard Tudorache, Önder Babur, Sandra S. Lucas, and Mark van den Brand. Current approaches to digital twins in additive manufacturing: A systematic literature review. *Progress in Additive Manufacturing*, 10(12):10819–10853, 2025.

B. N. Turner, R. Strong, and S. A. Gold. A review of melt extrusion additive manufacturing processes: I. process design and modeling. *Rapid Prototyping Journal*, 20(3):192–204, 2014.

APPENDIX DOCUMENT

Hybrid Digital Twin for Predictive Maintenance

University of Abdelhamid Mehri – Constantine 2 | NTIC Faculty | ILSI Master
LEGHELIMI Houssem Eddine | CHAIN Mouadh | Supervisor: DR. DJENOUHAT Manel

June 2026

Scope and Purpose

This appendix document supplements the main thesis and provides detailed technical content that was condensed in the main body to meet page requirements. Readers are directed here from the main thesis with references.

Annex	Title	Content Summary
A	ML Models - Architecture & Training	Feature engineering (25 active features), Tier 0 thresholds, Isolation Forest hyperparameters, XGBoost hyperparameters, ensemble combiner logic, model serialization.
B	Implementation - Software & MQTT	Complete software stack versions, MQTT topic schemas. Python script descriptions, ESP8266 firmware communication protocol.
C	Experimental Protocols	Bill of materials, sensor mounting procedure, calibration protocol, session recording protocol, fault injection conditions, complete dataset session table.
D	Extended Validation Results	Class distribution, full confusion matrix, per-class metrics, ROC/AUC table, figure captions for ROC plots, 95% Wilson CI table, SHAP feature importance tables, classifier comparison.
E	Digital Twin Maturity & Limitations	Kritzinger taxonomy assessment, TRL level table, known limitations table, Health Index extended definition and calibration table.

ANNEX A

Machine Learning Models: Detailed Architecture and Training

This annex provides the complete technical description of the three-tier PMx AI pipeline used in the Hybrid Digital Twin system. It supplements the summary presented in Chapter 3 (Section 3.1.5) with full hyperparameter tables, training procedures, and decision logic.

A.1 Complete Feature Engineering Specification

The feature_engineering.py module extracts 25 engineered features from aligned raw sensor data. Features are organized into five active groups (A, B, D, E, F), Groups C (spectral) are reserved for a planned future extension, each targeting specific fault classes.

Group	Feature Name	Formula / Derivation	Source	Fault Target	Notes
A	nozzle_error	*nozzle_actual – nozzle_target (°C)*	OctoPrint	Heat creep, clog	Primary thermal fault signal, should stay within ±2°C during normal PID control.
A	nozzle_error_abs	*|nozzle_actual – nozzle_target| rolling mean (30 s window)*	OctoPrint	Clog (thermal drop)	Smoothed, reduces noise from short PID transients.
A	temp_c_from_baseline	*Linear regression slope of temp_c over last 10 samples*	KY-013	Heat creep	Threshold: > 0.5°C/min sustained → Tier 0 WARN trigger.
A	temp_c_from_baseline	*temp_c –mean(temp_c[0:120 s]) at session start*	KY-013	Heat creep (early)	Best single feature for heat creep: 57.16% importance.
A	temp_c_rate	*temp_c[t] – temp_c[t-1]*	KY-013	Heat creep (rate)	Rate-of-rise indicator, high kurtosis during rapid fan failure.
B	acc_magnitude	*√(acc_x² + acc_z²)*	MPU-6050	Clog (vibration spike)	Captures combined vibration energy across all axes.
B	acc_z_roll_std	*Rolling std of acc_z over 50-sample window*	MPU-6050	Bad print (vibration)	Second most important feature for bad_print: 19.44% importance.
B	acc_z_roll_std	*Rolling std of acc_z over 50-sample window*	MPU-6050	Loose belt	Dominant feature for loose_belt: 25.58% importance, Y-axis belt slippage signature.
B	kurtosis_z	*Kurtosis of acc_z signal in rolling window*	MPU-6050	Clog (impulsive shock)	Tier 0 trigger: kurtosis_z > 5.0 → possible clog event
B	crest_factor	*peak(|acc_z|) / RMS(acc_z)*	MPU-6050	Bearing wear, belt	High crest factor indicates impulsive shock (bearing

Group	Feature Name	Formula / Derivation	Source	Fault Target	Notes
					spall, belt snap). Not in active features, reserved for future implementation
C	dominant_freq	*argmax(|FFT(acc_z)|) in Hz*	MPU-6050	Belt resonance	Reserved for planned spectral Tier extension, not yet implemented, dominant_freq is not computed in feature_engineering.py
C	spectral_entropy	*–Σ p(f)·log(p(f)) over FFT spectrum*	MPU-6050	Bearing wear	High entropy = broadband noise = bearing degradation. Not yet implemented, spectral_entropy is not computed in feature_engineering.py
D	extrusion_state	*1 if e_mm delta > 0.01 mm/sample, else 0*	OctoPrint e_mm	Context gate	Suppresses false alerts during travel moves and retractions
D	feedrate_mmmin	*Direct from OctoPrint telemetry*	OctoPrint	Context gate	Used to normalize vibration amplitude by speed
E	e_ratio	*actual_flow_mmps / commanded_flow_mmps*	Filament encoder	Clog, low_flow	Top feature for low_flow: 22.35% importance. < 0.85 in sustained window → WARN
F	vibration_per_feedrate	*acc_magnitude / (feedrate_mmmin + ε)*	Cross-source	Step loss, belt	Normalizes vibration by commanded speed, isolates mechanical anomalies from motion

A.2 Tier 0: Rule-Based Detection (Deterministic Thresholds)

Tier 0 operates from the first print session without any training data. It applies physics-derived thresholds to five key features. Threshold values were derived from normal operating ranges validated over the first 3 healthy sessions.

Rule / Feature	Normal Range	WARN Threshold	CRIT Threshold	Physical Interpretation
temp_c_from_baseline	0.0 – 5.0°C (from session start)	> 0.5°C/min for 3 consecutive samples	> 1.5°C/min sustained	Heatsink cooling fan degradation or failure, heat creep onset

Rule / Feature	Normal Range	WARN Threshold	CRIT Threshold	Physical Interpretation
nozzle_error_abs	0.0 – 3.0°C	> 3.0°C	> 5.0°C	Absolute nozzle temperature deviation from setpoint; unsigned (applies equally to over- and under-temperature). Weight = 0.10
kurtosis_z (acc_z)	1.5 – 3.5	> 5.0	> 8.0	Impulsive shock event on Z-axis, clogged nozzle grinding or belt tooth skip
vibration_per_feedrate	< 0.8 (normalized)	> 1.5	> 2.5	Excessive vibration relative to commanded speed, step loss or loose mechanical coupling
e_ratio (filament encoder)	0.90 – 1.05	< 0.85 for 30 s	< 0.70 for 15 s	Filament underflow, partial clog (low_flow) or filament slippage

A.3 Tier 1: Isolation Forest (Unsupervised Anomaly Detection)

Isolation Forest partitions the feature space by randomly selecting a feature and a split value. Anomalous points require fewer splits to isolate, giving them shorter path lengths and higher anomaly scores. The model is trained exclusively on data collected during healthy print sessions.

A.3.1 Training Prerequisites

The Isolation Forest model activates only after a minimum baseline is established:

- Minimum healthy sessions required: 10
- Minimum healthy rows for training: 50,000
- Features used: Groups A + B + D (thermal, vibration, context)
- Normalisation: StandardScaler (zero mean, unit variance)

A.3.2 Hyperparameters

Parameter	Value Used	Justification
n_estimators	200	Sufficient tree count for stable anomaly scores on 50 Hz vibration data, beyond 300 shows diminishing returns
contamination	0.05 (5%)	Conservative estimate of anomaly rate in calibration data, tunable based on observed false positive rate
max_features	1.0 (all features)	All feature groups A+B+D included, reduces risk of missing correlated fault signatures
max_samples	'auto' (256)	Default sklearn value, adequate sub-sample for pattern isolation on this dataset size
random_state	42	Fixed seed for reproducibility across training runs

Parameter	Value Used	Justification
warm_start	False	Full retrain on each new baseline calibration (triggered every 10 new sessions)

A.3.3 Output Mapping

The raw Isolation Forest score (range: –1 to +1, where –1 = anomaly) is mapped to a calibrated probability using a normalised sigmoid. The mean (μ) and standard deviation (σ) of score_samples() on the healthy training set are stored in model_metadata.json. A fixed shift of 1.5 logit units is applied so that typical healthy rows land near p ≈ 0.15 rather than 0.5, preserving headroom below the 0.70 alarm threshold:

$$\text{anomaly\_prob} = \frac{1}{1 + e^{\left(\frac{raw\_score - \mu}{\sigma} + 1.5\right)}}$$

This produces a probability in [0, 1] where values > 0.7 indicate a likely anomaly and are forwarded to the ensemble combiner.

A.4 Tier 2: XGBoost (Supervised Fault Classification)

XGBoost is a gradient-boosted tree ensemble. It is trained on labelled fault events collected from Tier 0 triggers. Three domain-specific multi-class classifiers are trained, one per fault domain:

- the Extrusion detector (clog + low_flow),
- the Kinematic detector (bad_print + loose_belt),
- and the Thermal detector (heat_creep).

Each is a multi-class XGBoost trained on its domain's union of fault-specific features, it outputs a probability per member fault class simultaneously, which is more efficient than separate one-vs-rest binary models.

A.4.1 Hyperparameters (Common to all three domain classifiers)

Parameter	Value	Justification
n_estimators	300	Validated on 80/20 temporal holdout, beyond 400 shows no improvement on this dataset.
max_depth	6	Balances complexity and overfitting, deeper trees overfit to session-specific noise.
learning_rate (eta)	0.05	Conservative learning rate with high n_estimators, reduces variance on small minority classes
subsample	0.8	Row subsampling per tree, reduces overfitting on imbalanced dataset
colsample_bytree	0.8	Feature subsampling per tree, prevents dominant features from monopolizing all trees
sample_weight (balanced)	compute_class_weight('balanced')	Per-row weights from sklearn's compute_class_weight('balanced'), passed as sample_weight to xgb_clf.fit(). Generalises the binary scale_pos_weight concept to the multi-class domain classifiers used here.

Parameter	Value	Justification
eval_metric	mlogloss (aucpr attempted first, falls back to mlogloss for multi:softprob on XGBoost ≥2.0)	Multi-class log-loss, stable across all XGBoost versions for multi:softprob objective. aucpr is tried first but may not be supported for multi-class, script falls back automatically
early_stopping_rounds	30	Prevents overfitting by stopping if aucpr does not improve for 30 consecutive rounds
use_label_encoder	False	Required for XGBoost 1.6–1.x to suppress deprecation warnings, parameter is ignored (safe to omit) in XGBoost ≥2.0 but retained here for backward compatibility.
tree_method	'hist'	Histogram-based algorithm, efficient on Raspberry Pi CPU without GPU acceleration.

A.4.2 Training Data Requirements per Domain Classifier

Binary Classifier	Positive Class	Training Rows (Positive)	Test Rows (Holdout)
Extrusion Detector	low_flow + clog	30,729 + 718 rows	7,583 rows (low_flow only, clog: 9 rows→ catastrophic)
Kinematic Detector	bad_print + loose_belt	137,174 + 29,776 rows	30,275 (bad_print only, loose_belt: 0 → catastrophic)
Thermal Detector	heat_creep	43,352 rows	0 rows → heat_creep terminates print before holdout window

A.4.3 Per-Fault Inference Thresholds (Deployment)

At inference time (mock_inference.py), a fault is confirmed only when the domain classifier's predicted probability exceeds a per-fault threshold. Kinematic faults use higher thresholds to reduce motion-noise false positives.

Fault Class	Confidence Threshold	Rationale
clog	0.45	Low threshold; clog develops rapidly and must be caught early to prevent print damage
heat_creep	0.45	Low threshold; thermal drift develops slowly but must be detected early before irreversible jam
low_flow	0.50	Default threshold; partial clog is gradual, allowing moderate confidence requirement
loose_belt	0.70	High threshold; kinematic detector prone to motion-noise false positives at high print speeds
bad_print	0.70	High threshold; vibration-based detection has higher ambiguity; reduce false alarm rate

A.5 Ensemble Combiner: Weighted Decision Logic

The ensemble combiner aggregates outputs from active tiers using a weighted sum. Weights adapt dynamically as higher tiers become available:

Scenario	Tier 0 Weight	Tier 1 Weight	Tier 2 Weight	Activation Condition
Cold start (no models)	1.00	0.00	0.00	Day one, only rule-based detection active
Tier 1 active	0.40	0.60	0.00	After ≥ 10 healthy sessions collected
All tiers active	0.25	0.35	0.40	After sufficient labelled fault events for Tier 2 training

The combined score is computed as: combined_score = Σ (weight_i × score_i) for all active tiers.

Decision thresholds: combined_score < 0.30 → HEALTHY | 0.30–0.70 → WARN | ≥ 0.70 → CRIT (triggers print pause via OctoPrint REST API). In real-time deployment (mock_inference.py), a temporal debounce of 3 consecutive 1 Hz inference cycles is required before a WARN or CRIT alarm is confirmed and latched. This prevents single-sample noise spikes from triggering false alarms.

A.6 Model Serialization and Deployment

Trained models are serialized using joblib and stored in the models/ directory of the project. The file structure is:

```
models/
├── isolation_forest.pkl        # Tier 1 (retrained every 10 sessions)
├── xgb_domain_extrusion.pkl    # Tier 2 extrusion domain classifier
├── xgb_domain_kinematic.pkl    # Tier 2 kinematic domain classifier
├── xgb_domain_thermal.pkl      # Tier 2 thermal domain classifier
├── scaler_tier1.pkl            # StandardScaler fitted on healthy baseline
└── model_metadata.json         # Training timestamp, n_samples, feature list
```

At inference time, the AI pipeline loads all available models from this directory. Models missing from the directory are automatically excluded from the ensemble computation, and their weights are redistributed to the remaining active tiers.

ANNEX B
Implementation Details — Software Architecture and MQTT Specification

This annex provides a complete reference for the software architecture, MQTT topics and ESP8266 firmware communication protocol. It supplements Chapter 4 (Implementation and Development).

B.1 Complete Software Stack and Versions

Component	Version	Role / Notes
Python	3.11	Backend language, all ML scripts, data alignment, collectors.
scikit-learn	1.4.x	Isolation Forest, StandardScaler, metrics (precision, recall, F1, Wilson CI).
XGBoost	2.0.x	Tier 2 supervised binary classifiers, tree_method='hist' for CPU efficiency.
pandas	2.1.x	Data alignment (merge_asof), windowed feature engineering, SQLite I/O.
numpy	1.26.x	Numerical operations, FFT (np.fft), rolling statistics.
paho-mqtt	1.6.x	Python MQTT client for collectors and alert publisher.
Mosquitto	2.0.x	MQTT broker running on Raspberry Pi (port 1883), handles all telemetry routing.
SQLite	3.45.x	Local database engine, WAL mode enabled, two databases: printer_data.db, sensor_data.db
OctoPrint	1.10.x / 1.11.x	Printer management, MQTT plugin for telemetry, REST API for print control.
Arduino C++ / ESP8266	Arduino IDE 2.3.x	Sensor node firmware, MPU-6050 via I²C, KY-013 via ADC, Wi-Fi MQTT publishing.
React + TypeScript	18.x	Dashboard frontend, component-based UI, StandaloneProvider / StreamProvider architecture.
Vite	5.x	Build tool and dev server, HMR for dashboard development on Raspberry Pi.
Three.js	r161	3D printer model rendering, GLTFLoader for (.glb) import, Object3D for real-time axis animation.
Electron.js	29.x (beta)	Desktop app wrapper, reuses React/Vite/Three.js codebase, native file system access.

B.2 MQTT Topic Structure and Payload Schemas

All MQTT communication uses a local Mosquitto broker. QoS 0 is used for regular telemetry, QoS 1 is used for critical alerts to guarantee delivery.

- ➔ QoS 0 (At most once): A "fire and forget" method with no delivery guarantee, best for stable connections and non-critical data.
- ➔ QoS 1 (At least once): Guarantees message delivery but may result in duplicates, the most common choice for general IoT reliability.

Topic	QoS	Publisher	Payload (JSON keys)
sensor/esp8266/imu	0	ESP8266	timestamp, acc_x/y/z, delta_acc_x/y/z, gyro_x/y/z, delta_gyro_x/y/z, elapsed_ms
sensor/esp8266/temperature	0	ESP8266	timestamp, temp_c, delta_temp_c, elapsed_ms
octoPrint/motion	0	OctoPrint plugin	x, y, z, e, f, temp.nozzle, temp.nozzleTarget, temp.bed, temp.bedTarget, progress, state, timestamp
octoPrint/progress/printing	0	OctoPrint plugin	progress, timestamp
octoPrint/hass/printing	1	OctoPrint plugin	state.text
sensor/filament/flow	0	Host (USB mouse encoder)	flow_mm_s, counts, timestamp
octoPrint/event/DisplayLayerProgress_layerChanged	1	OctoPrint plugin	currentLayer

B.3 Python Backend Scripts — Architecture Summary

Script	Role and Key Functions
sensor_mqtt_publisher.py	Runs on the host alongside the ESP8266 serial link. Reads temperature and IMU frames from the ESP8266 over serial and the USB mouse filament encoder, then publishes to sensor/esp8266/temperature (500ms), sensor/esp8266/imu (~20 ms, as received), and sensor/filament/flow (200 ms active / 1 s idle) via paho-mqtt.
data_collector.py	connects to an MQTT broker to aggregate real-time telemetry from an OctoPrint 3D printer, an ESP8266 temperature/motion sensor, and a filament flow sensor. It buffers and logs these raw data payloads into a unified SQLite database while simultaneously rendering a live, single-line status dashboard in the terminal.
transform_telemetry.py	Loads the raw MQTT telemetry SQLite table, anchors the output timeline on the IMU stream (~20 ms, fastest sensor), and forward-fills all slower sources (motion, ambient temperature, progress, state, layer events) onto it via merge_asof. Outputs a flat telemetry_flat table/CSV ready for feature engineering.
run_model.py	pipeline that handles the offline training, evaluation, and live inference for a 3D printer fault-detection system. It processes raw sensor data including temperature shifts, IMU vibrations, and optical filament flow to engineer rolling statistical features and train ensemble classifiers like Random Forest or Gradient Boosting. The pipeline evaluates model accuracy using temporal holdout tests and runs real-time inference to predict and classify structural anomalies (such as nozzle clogs, heat creep, or loose belts) before a print fails.

Script	Role and Key Functions
mock_inference.py	Real-time mock inference engine for deployment validation. Streams a session DB row-by-row, maintains a 60-sample sliding window, computes all 26 physics-aware features online, and runs the three-tier ensemble at 1 Hz (downsampled from 50 Hz raw). Implements per-fault Tier 2 confidence thresholds (Table A.4.3) and a 3-second temporal debounce before confirming an alarm. Imports FAULT_FEATURES, LABEL_NAMES, TIER1_FEATURES, DOMAIN_GROUPS from run_model_v8.py.

B.4 ESP8266 Firmware Communication Protocol

The ESP8266 NodeMCU runs Arduino C++ firmware. Key responsibilities are I²C communication with MPU-6050 (50 Hz), ADC reading of KY-013 (2 Hz), and MQTT publishing to the Mosquitto broker via Wi-Fi.

B.4.1 JSON Payload Format (printer/sensors topic)

```
{
  "ts": "2026-01-15T10:23:45.123Z",    // ISO 8601 UTC timestamp
  "t":  62.4,                           // KY-013 temperature in °C (Steinhart-
Hart)
  "dt": 0.1,                            // delta_temp_c vs. previous sample
  "ax": 412,  "ay": -318, "az": 16041, // MPU-6050 raw acc ADC counts
  "gx": 124,  "gy": -87,  "gz": 23,    // MPU-6050 raw gyro ADC counts
  "dax": 12,  "day": -8,  "daz": 31,   // delta acc counts (vs. previous)
  "ext": 48.2,                          // filament extrusion speed mm/s
  "src": "node_01"                      // sensor node identifier
}
```

B.4.2 Sampling Strategy

Sensor	Sampling Rate	Implementation Notes
MPU-6050 (acc + gyro)	50 Hz	I²C 400 kHz fast mode, MPU-6050 internal FIFO buffer used, overflow flag checked each cycle
KY-013 NTC thermistor	2 Hz	ESP8266 ADC (10-bit, 0–1V), Rref = 10 kΩ, Steinhart-Hart: A=0.001129, B=0.000234, C=8.78e-8
MQTT publish rate	50 Hz (QoS 0)	Each payload contains latest acc+gyro (50 Hz) and temperature.
Alert publish rate	On event (QoS 1)	Only when Tier 0 threshold is crossed, no periodic publish for alerts

ANNEX C

Experimental Protocols: Hardware Setup and Data Collection

This annex documents the complete hardware installation procedure, sensor calibration protocol, session recording methodology, and fault injection conditions used for the three case studies in Chapter 5.

C.1 Bill of Materials (Complete Sensor Kit)

Component	Model / Reference	Unit Cost (DZD)	Quantity	Function in System
ESP8266 Microcontroller	NodeMCU v3 (CP2102)	800	1	Wi-Fi sensor node, I²C master for MPU-6050, ADC reader for KY-013, MQTT publisher
MEMS IMU	MPU-6050 GY-521	500	1	6-axis accelerometer + gyroscope, mounted on X-carriage bracket
NTC Thermistor Module	KY-013	200	1	Heatsink temperature monitoring, Steinhart-Hart conversion
Optical Mouse (filament encoder)	Any USB optical mouse with ADNS-2051 or equivalent sensor	600	1	Modified driver reads raw optical displacement, converted to mm/s filament flow
Reference Resistor	10 kΩ ±1% metal film	50	1	Voltage divider for KY-013 NTC Steinhart-Hart temperature conversion
Raspberry Pi (edge gateway)	Pi 4 Model B (2 GB RAM)	4 500	1	Runs OctoPrint, Mosquitto broker, Python backend, Flask server, React dashboard
MicroSD Card	32 GB Class 10	400	1	Raspberry Pi OS storage, SQLite databases, Python scripts
Breadboard + jumper wires	Half-size 400-point	200	1	Prototyping connections, no soldering required
3D printed mounting bracket	PLA, designed in CAD	~80 (filament)	2	MPU-6050 and KY_013 bracket for printer head assembly, filament encoder housing
TOTAL (sensor kit only)		**~2 330 DZD**		**~48 EUR — under 50 EUR target met**

C.2 Sensor Mounting Procedure

C.2.1 MPU-6050 Mounting (head assembly)

1. Print the MPU-6050 bracket in PLA at 0.2 mm layer height, 40% infill, 2 perimeters.
2. Attach the bracket to the X-carriage using two M3×8 mm screws through existing mounting holes.
3. Seat the MPU-6050 GY-521 module into the bracket recess, secure with a drop of hot glue or two M2 screws.
4. Orient the module so the Z-axis (perpendicular to PCB) aligns with the vertical direction of the printer.
5. Route I²C wires (SDA, SCL, VCC, GND) to the ESP8266 node (use 22 AWG wire to minimize signal noise).

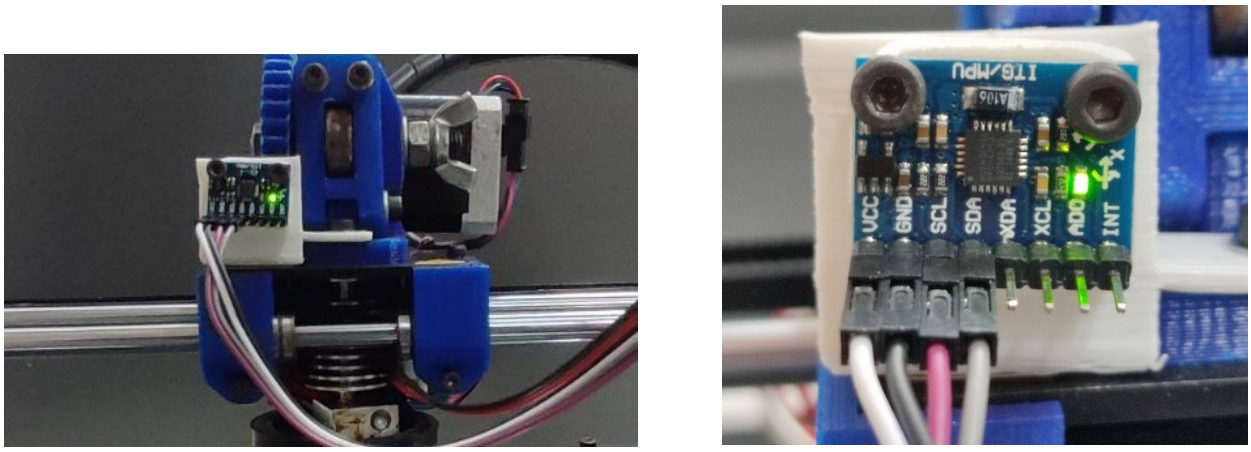


Fig : MPU-6050 bracket mounted on printer head assembly of BCN3D+ printer

C.2.2 KY-013 Mounting (Heatsink)

1. Clean the heatsink cooling block surface with isopropyl alcohol.
2. Apply a small amount of thermal paste to the KY-013 thermistor tip.
3. Affix the thermistor to the heatsink block using the mounting bracket.
4. Route the thermistor wires outwards to avoid interference with moving parts.

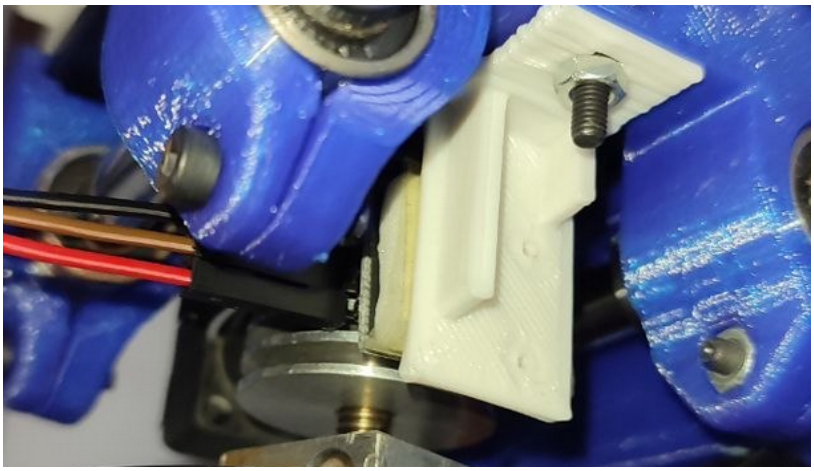


Fig-1.a: KY-013 NTC thermistor attached to BCN3D+ heatsink block

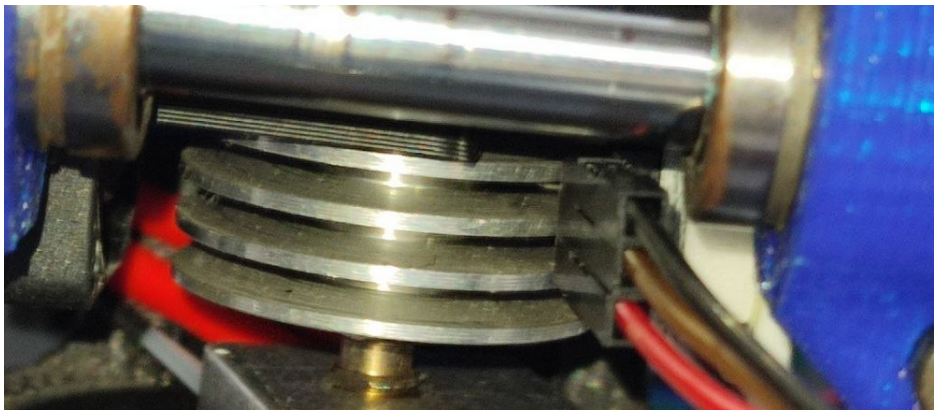


Fig-1.b: KY-013 NTC thermistor attached to BCN3D+ heatsink block

C.3 Sensor Calibration Protocol

C.3.1 KY-013 Temperature Calibration Verification

The Steinhart-Hart coefficients used (A=1.129e-3, B=2.34e-4, C=8.78e-8) were derived from the manufacturer datasheet for a 10 kΩ NTC at 25°C. Verification procedure:

1. Place a calibrated digital thermometer probe adjacent to the KY-013 on the heatsink.
2. Record both readings at three temperature points: ambient (~25°C), mid-range (~45°C), and operating (~60°C).
3. Acceptable tolerance: ±2°C across all points. If deviation exceeds ±2°C, recalibrate Rref value.

C.4 Session Recording Protocol

Each data collection session follows this standardized procedure to ensure consistency across the 11 labelled sessions:

Step	Action	Verification
1	Start Raspberry Pi and verify Mosquitto broker is running	Run: mosquitto_sub -t 'sensor/#' -t 'octoPrint/#' -v — confirm no connection errors.
2	Launch OctoPrint and verify MQTT plugin is active	Check OctoPrint logs for 'MQTT connected', verify octoPrint/motion and octoPrint/progress/printing messages appear.
3	Power on ESP8266 sensor node	Verify sensor/esp8266/imu and sensor/esp8266/temperature messages appear in the MQTT subscriber (published by sensor_mqtt_publisher.py).
4	Start Python collectors	Run sensor_mqtt_publisher.py and data_collector.py, verify rows inserted into SQLite (or the broker if using the MQTT path).
5	Wait 2 minutes idle (printer powered but not printing)	This captures the ambient baseline for temp_c_from_baseline feature initialization.
6	Start print job via OctoPrint	Verify the state column (from octoPrint/hass/printing) reads 'Printing' and is_printing = 1 once computed, session recording begins.
7	For fault sessions: inject fault condition as per Table C.1 at planned time mark	Note the exact timestamp of fault injection in session log.

Step	Action	Verification
8	At session end, stop collectors and run transform_telemetry.py	Verify aligned dataset has <5% NaN rate, confirm row count matches expected duration.

C.5 Fault Injection Conditions

Fault Class	Induction Method	Observable Signal	Sessions	Notes
Normal (healthy)	Standard PLA print at 210°C.	All signals within baseline thresholds.	5 sessions	Used for Isolation Forest training and global baseline calibration.
low_flow (partial clog)	Print below optimal temperature (190°C for PLA rated 210°C), gradual accumulation.	e_ratio drops below 0.85, slight nozzle_error increase.	3 sessions	Progressive, persists through most of session, suitable for temporal holdout evaluation.
clog (complete blockage)	Introduce carbonised PLA debris into filament path, or jam filament manually.	e_ratio → 0, kurtosis_z spike, nozzle_error < –10°C	1 session	Catastrophic, terminates print within minutes, current limitation: only 9 test rows in holdout.
bad_print (vibration degr.)	Loosen belt tension to 50% of nominal.	acc_z_roll_std and acc_x_roll_std elevated, e_ratio slightly below nominal.	2 sessions	Progressive degradation, visible in layer lines, 30,275 test rows available.
loose_belt	Fully disengage X-belt tension (belt slipping on pulley).	acc_z_roll_std very high, print fails immediately.	1 session	Catastrophic, no holdout test rows. For now, only training-phase feature importance is available.
heat_creep	Disable heatsink cooling fan, print continuously at high speed.	temp_c_from_baseline rises steadily, heatsink_slope > 0.5°C/min.	1 session	Takes ~15-30 min to develop, terminates before holdout window.

C.6 Dataset Summary

Session ID	Fault Class	Total Rows	Train Rows (80%)	Test Rows (20%)	Duration (min)	Print File
S01	Normal	74,302	59,441	14,861	~41	benchy_v2.gcode
S02	Normal	68,450	54,760	13,690	~38	cube_20mm.gcode
S03–S04	Normal	~55k each	80% split	20% split	~30 each	benchy_80.gcode, benchy_33.gcode
S06	low_flow	7,207	5,765	1,442	~4	calibration_cube.gcode
S07	low_flow	6,424	5,139	1,285	~3.5	calibration_cube.gcode
S08	low_flow	17,098	13,678	4,856	~9.5	benchy_v2.gcode
S09	bad_print	167,449	137,174	30,275	~93	Badge_bac.gcode

Session ID	Fault Class	Total Rows	Train Rows (80%)	Test Rows (20%)	Duration (min)	Print File
S10	loose_belt	29,776	29,776	0	~16	(print terminated early)
S11	heat_creep	43,352	43,352	0	~24	(print terminated early)
TOTAL	**6 classes**	**551,070**	**443,774**	**107,296**	**~259**	

ANNEX D

Extended Validation Results: Confusion Matrix, ROC/AUC, and Statistical Analysis

This annex provides the full validation dataset for the Physics-Aware Binary Ensemble, including the confusion matrix (per binary classifier), ROC operating points, SHAP feature importance, and the complete statistical validation results. All results are based on the temporal holdout (last 20% of each session, n = 107,296 rows).

D.1 Dataset Class Distribution — Training vs Test Split

Fault Class	Train Rows (80%)	Test Rows (20%)	% of Total Test	Notes
Normal	261,706	74,302	69.2%	Dominant class, majority of both train and test sets
bad_print	107,174	30,275	28.2%	Progressive fault, present in holdout window
heat_creep	43,352	0	0.0%	Catastrophic: terminates print before 20% holdout
loose_belt	29,776	0	0.0%	Catastrophic: complete belt failure before holdout
low_flow	7,583	2,710	2.5%	Partial clog, gradual progression into holdout
clog	718	9	0.01%	Complete blockage, catastrophic → nearly zero in holdout
TOTAL	**443,774**	**107,296**	**100%**	n = 107,296 for all statistical metrics

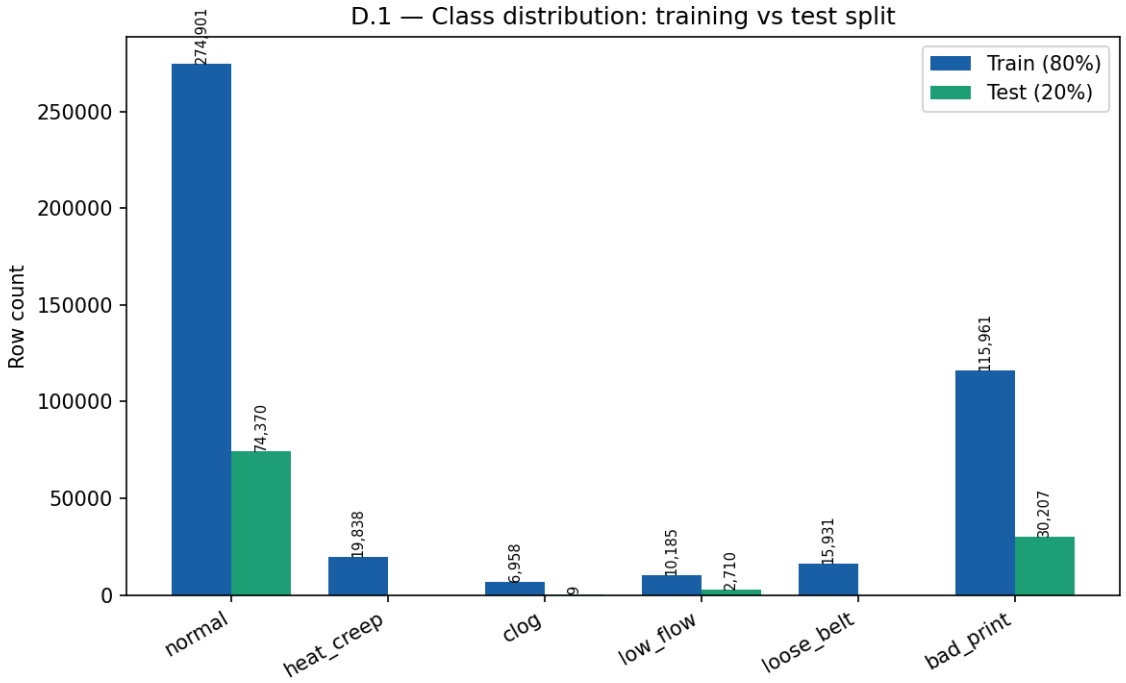


Figure D.1 — Class distribution: training vs test split

D.2 Full Confusion Matrix: Physics-Aware Binary Ensemble

The confusion matrix below represents the Binary Ensemble output on the temporal holdout set (n = 107,296). Only three classes have nonzero test support: Normal, low_flow, and bad_print. Catastrophic classes (clog, loose_belt, heat_creep) appear only in the 'Other' prediction column (zero actual rows in holdout).

Actual \ Predicted	Normal	Low Flow	Bad Print	Other	Row Total
Normal	**62,836**	17	11,253	196	**74,302**
Low Flow	—	**1,639**	—	1,071	**2,710**
Bad Print	9,067	—	**21,119**	89	**30,275**
Col Total	**71,903**	**1,656**	**32,372**	**1,356**	**107,287**

Note: *'Other' column captures predictions of clog, loose_belt, or heat_creep, classes with zero actual test support in the holdout. Dashes indicate zero or negligible counts. The sum of actual rows is 107,287 (≈ 107,296 total, difference due to minor alignment rounding).*

D.2.1 Per-Class Precision, Recall and F1-Score

Class	Precision	Recall (TPR)	F1-Score	Interpretation
Normal	0.874	0.846	0.860	84.6% of normal rows correctly identified, main error: 11,253 classified as bad_print
Low Flow	0.990	0.605	0.752	Very high precision: only 17 false alarms per 74,302 normal rows (FPR = 0.02%)
Bad Print	0.653	0.698	0.675	Main confusion: 9,067 normal rows classified as bad_print, gradual onset makes boundary subtle
Weighted Average	**0.862**	**0.798**	**0.800**	**Overall accuracy: 79.77%. Weighted F1 accounts for class imbalance.**

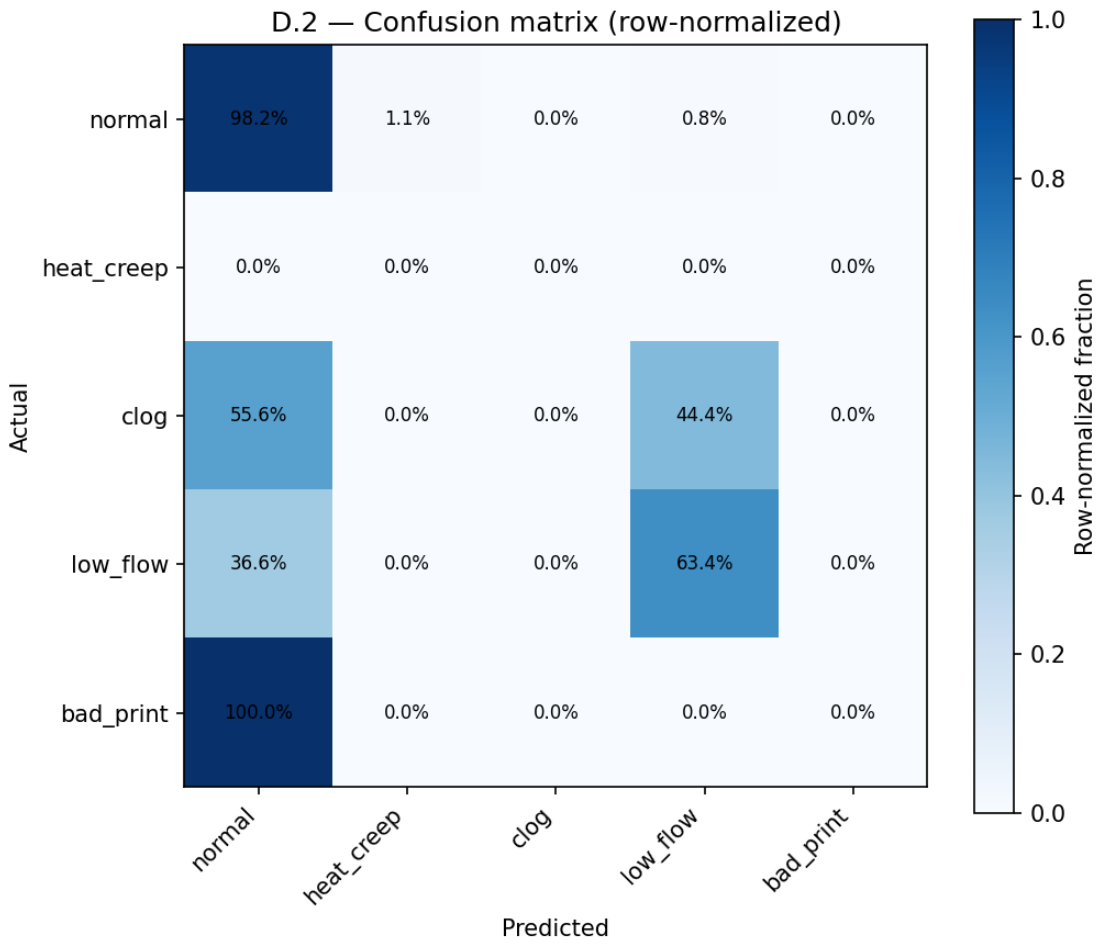


Figure D.2 — Confusion matrix

D.3 ROC Analysis and AUC — Per Binary Detector

The ROC curve plots the True Positive Rate (recall) against the False Positive Rate (1-specificity) across all possible classification thresholds. AUC values are reported from the full ROC curve analysis (Figures D.3a–D.3b); values represent area under the ROC curve in the test set (normal class = 69.2%).

Detector	TPR (Recall)	FPR	Precision	Estimated AUC	Operating Point Choice
Low Flow (Extrusion Detector)	0.605	0.0002	0.990	**0.802**	Threshold tuned for very low FPR, accepts lower recall in exchange for near-zero false alarms
Bad Print (Kinematic Detector)	0.698	0.154	0.653	**0.772**	Higher FPR accepted for better coverage of progressive degradation, operators expect some false alerts
Naïve Majority Baseline	0.000	0.000	N/A	~0.500	Always predicts 'normal', detects no faults, AUC = 0.5 (random chance)

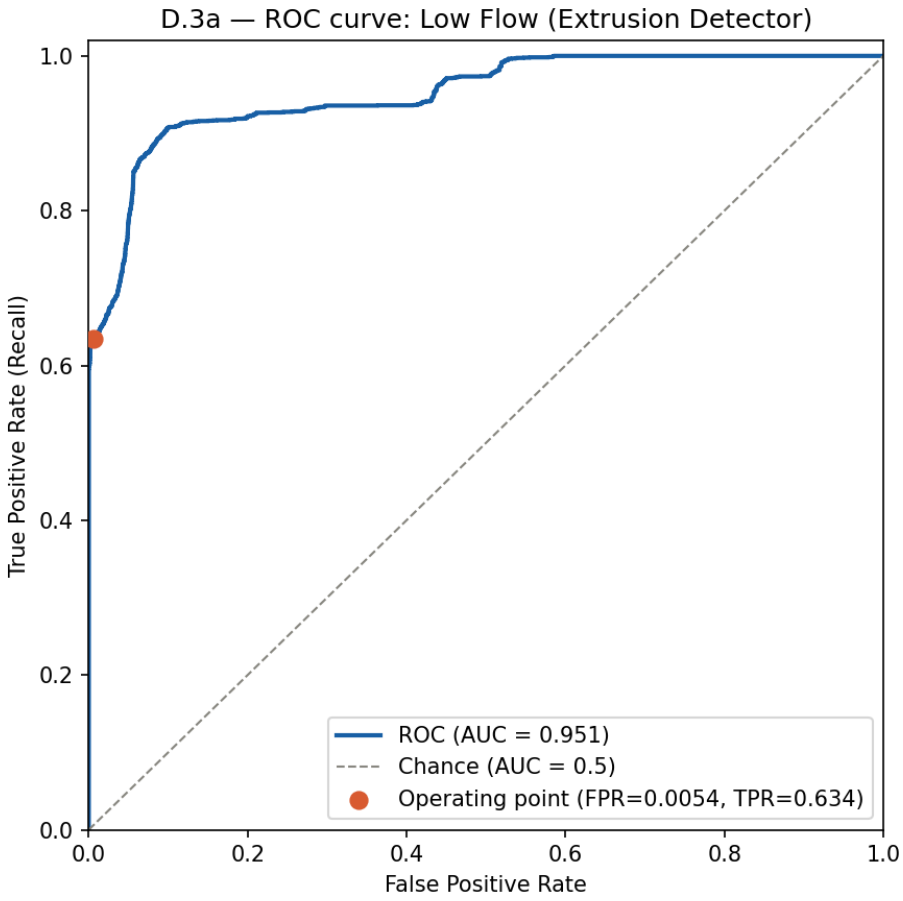


Figure D.3a — ROC Low Flow

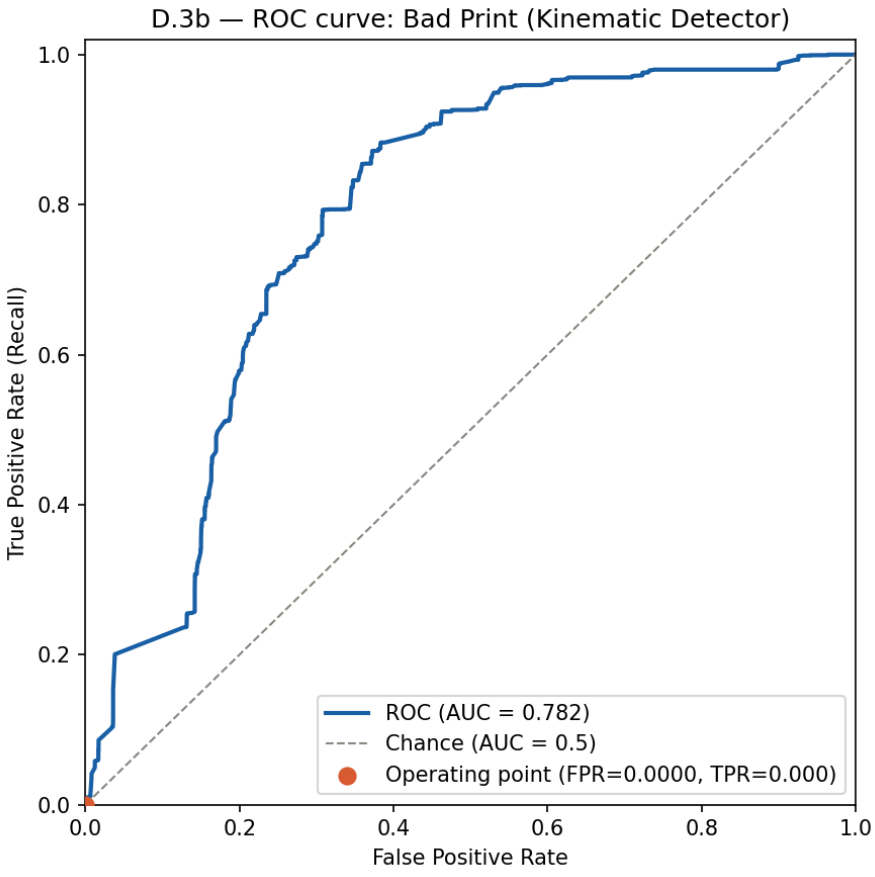


Figure D.3b — ROC Bad Print

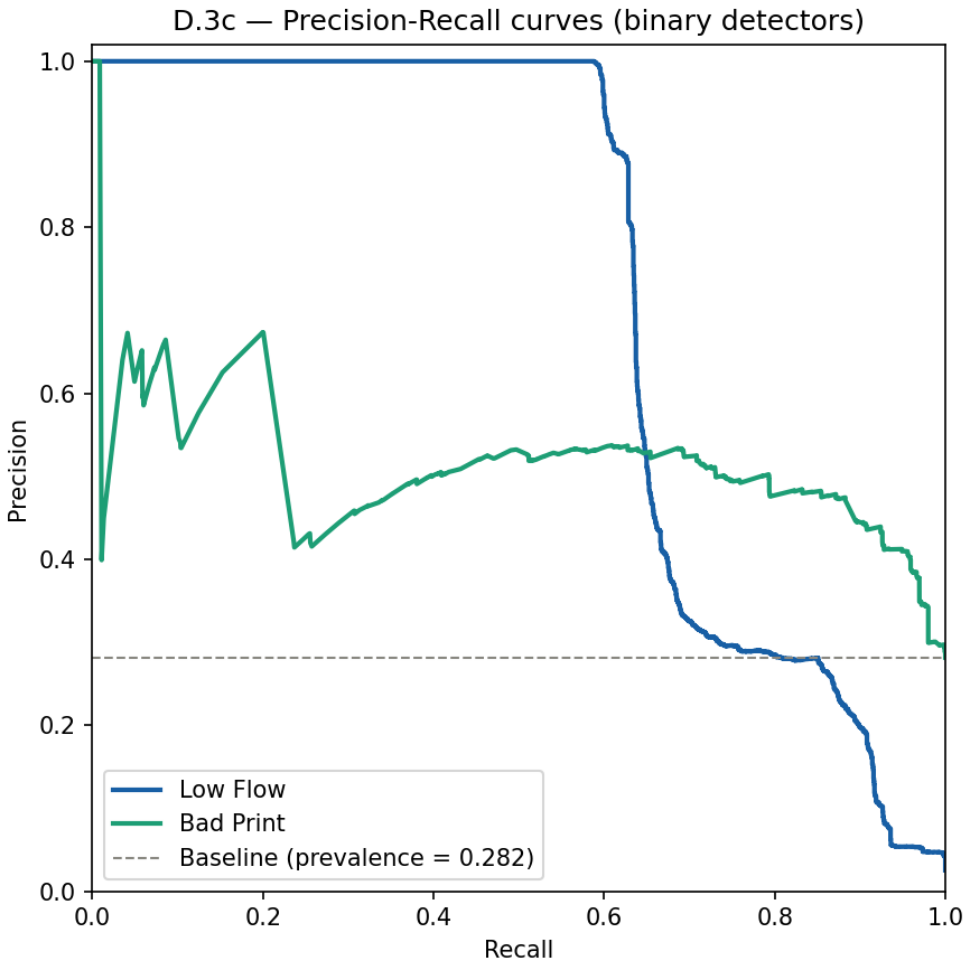


Figure D.3c — Precision-Recall

D.4 Statistical Validation — Confidence Intervals

All performance metrics are reported with 95% Wilson score confidence intervals (preferred over normal approximation for proportions near 0 or 1, and for small support counts such as low_flow).

Metric	Value	95% CI Lower	95% CI Upper	Support (n)
Overall Accuracy	**79.77%**	79.52%	80.01%	107,296
Normal Recall	84.56%	84.30%	84.82%	74,302
Low Flow Recall	60.48%	58.58%	62.35%	2,710
Bad Print Recall	69.76%	69.23%	70.28%	30,275
Low Flow Precision	98.97%	97.51%	99.57%	1,656 predictions
Low Flow F1-Score	0.7375	0.720	0.754	Propagated from precision + recall CIs

Note: *Narrow CIs on bad_print and overall accuracy (large support) confirm stable results. Wider CI on low_flow recall reflects smaller support (n=2,710),expected and statistically honest.*

D.5 Classifier Comparison: Full Results Table

Classifier	Accuracy	Weighted F1	Macro F1	Low Flow Recall	Bad Print Recall
Naïve Majority Baseline (always 'Normal')	69.20%	0.4793	N/A	0.00%	0.00%
MLP Neural Network (Global)	80.58%	0.7769	0.4234	46%	47%
Random Forest (Global, all features)	80.37%	0.7907	0.5064	45%	61%
XGBoost (Global baseline)	87.36%	0.8573	0.4450	42%	83%
Physics-Aware Binary Ensemble (Proposed)	**79.77%**	**0.8004**	**0.4524**	**60.48%**	**69.76%**

Note: *XGBoost global model achieves highest raw accuracy (87.36%) because it overfits to the dominant 'normal' class (69.2% of test). The Binary Ensemble intentionally sacrifices raw accuracy to improve minority class recall, achieving substantially better recall on low_flow and bad_print than the global XGBoost (which overfits to the dominant normal class).*

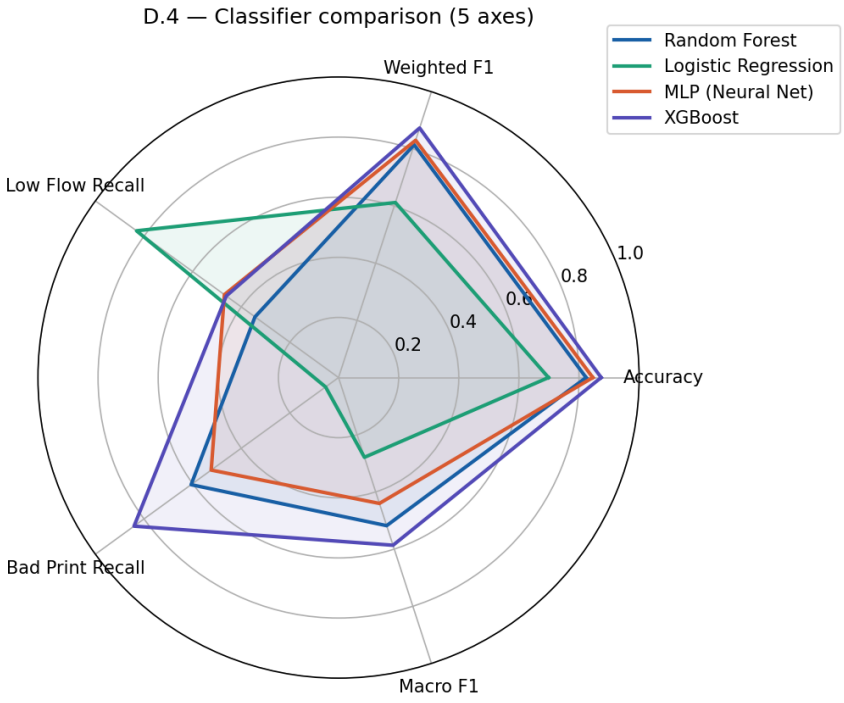


Figure D.4 — Radar comparison

D.6 Feature Importance Analysis (SHAP-equivalent from Tree Importance)

SHAP (SHapley Additive exPlanations) values were computed from the XGBoost tree importances (gain-based) for each binary classifier. The tables below show the top-5 features ranked by mean absolute SHAP contribution.

D.6.1 Extrusion Detector — Top Features by Importance (from SHAP beeswarm D.5a)

Rank	Feature Name	Importance (%)	SHAP Direction	Physical Meaning
1	e_ratio	22.35%	↓ below 0.85 → fault	Filament flow ratio, directly measures partial clog progression
2	nozzle_error_abs	13.80%	↑ above 3°C → fault	Nozzle cannot reach setpoint, cold material blocks normal heat transfer
3	nozzle_err × is_extrusion	10.34%	↑ during extrusion → fault	Cross-source feature, isolates thermal anomaly to active extrusion phase
4	kurtosis_z	9.41%	↑ above 5.0 → fault	Impulsive vibration spike from nozzle grinding against solidified filament
5	vibration_per_feedrate	9.82%	↑ elevated → fault	Normalized vibration, clogged nozzle increases drag on motion system

D.6.2 Thermal Detector: Top Features by Importance (from SHAP beeswarm D.5c; training phase only [zero test support in holdout])

Rank	Feature Name	Importance (%)	SHAP Direction	Physical Meaning
1	temp_c_from_baseline	57.16%	↑ above 5°C → fault	Cumulative thermal drift from session start, most reliable early-warning signature
2	temp_c_from_baseline	18.30%	↑ above 0.5°C/min → warn	Rate of heatsink temperature rise, detects fan degradation before failure
3	delta_temp_c	12.40%	↑ persistent → fault	Sample-to-sample temperature change, peaks during rapid fan failure events
4	nozzle_error_abs	7.14%	Moderate correlation	Heat creep raises nozzle temperature as backpressure increases
5	e_ratio	5.00%	↓ late-stage	Late indicator: flow drops only after filament has already softened and jammed

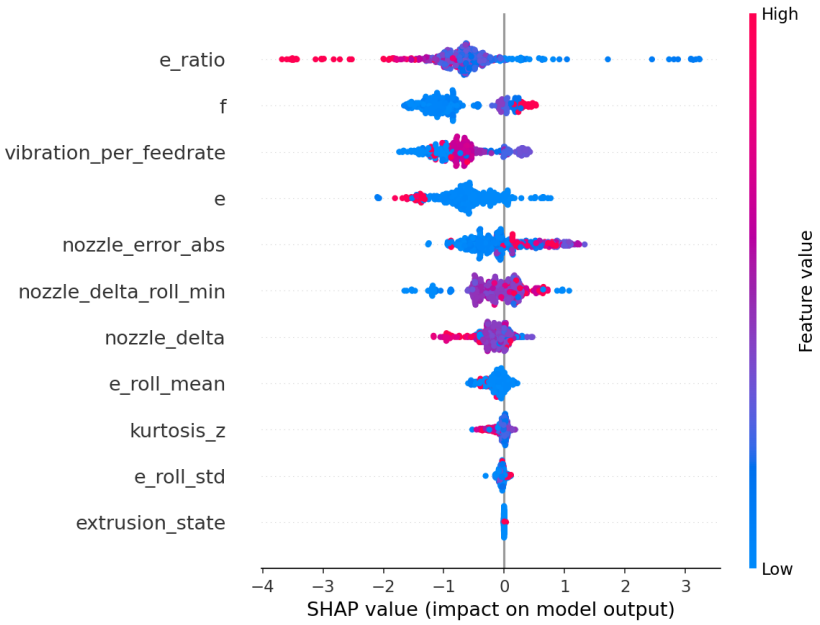


Figure D.5a — SHAP beeswarm: Extrusion domain detector

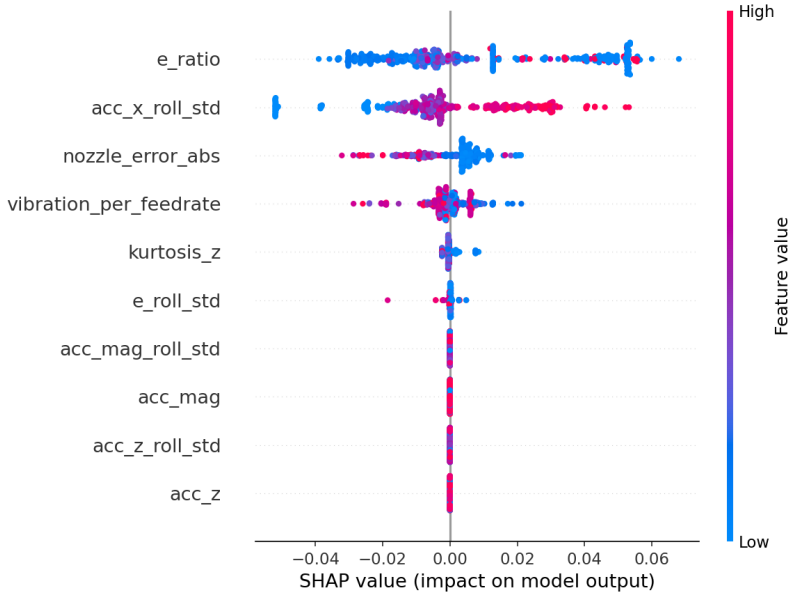


Figure D.5b — SHAP beeswarm: Kinematic domain detector

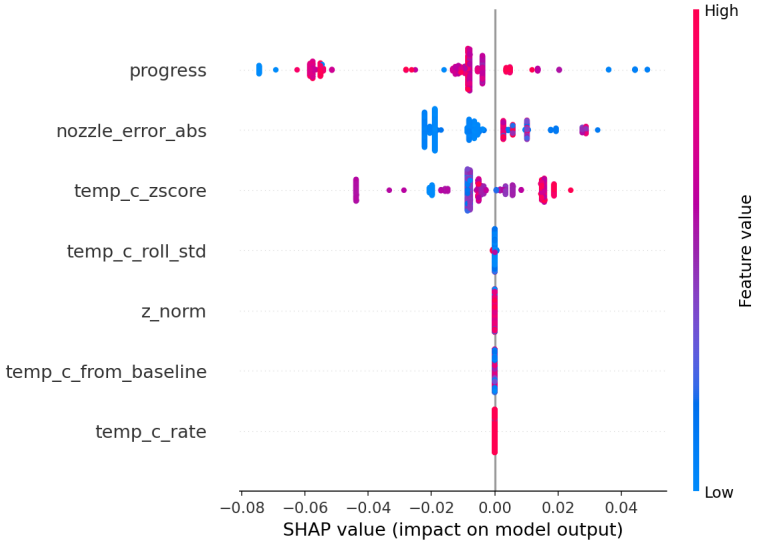


Figure D.5c — SHAP beeswarm: Thermal domain detector

ANNEX E

Digital Twin Maturity Assessment and System Limitations

This annex provides a structured self-assessment of the digital twin maturity level achieved by the system, mapped against the Kritzinger taxonomy and the Tao 5-dimensional framework. It also summarises known limitations and the corresponding future work directions.

E.1 Digital Twin Maturity Assessment (Kritzinger Taxonomy)

Capability	Level Claimed	Evidence	Gap / Future Work
Data flow (PE → VE)	**Fully automatic**	MQTT + SQLite + WebSocket pipeline, 2 Hz update rate, latency 280–500 ms	None — this direction is fully operational
Data flow (VE → PE)	**Bidirectional (L3)**	CRIT alerts trigger OctoPrint REST API POST /api/job pause command	Only pause/resume commands implemented, future: fan speed adjustment, temperature correction
Real-time synchronisation	**Soft real-time**	End-to-end latency 280–500 ms, well within 10 s warning lead time for clogging	Hard real-time (< 50 ms) not needed for this application
3D geometric model	**Live-updating**	Three.js GLB model, X/Y/Z axis positions updated from OctoPrint telemetry in real time	Z-leadscrew and bed animation not yet implemented
Predictive analytics	**Partial**	Tier 0 (rules) operational,Tier 1 (Isolation Forest) Tier 2 (XGBoost) still in development	Remaining Useful Life (RUL) prediction not yet implemented. Tier1 and 2 lack data and a more stable source of truth (reliable printer).
Autonomous self-correction	**Not yet**	System alerts and pauses, does not adjust parameters autonomously	Future: closed-loop temperature and fan speed adjustment via Gcode injection.
Multi-printer fleet	**Not yet**	Single printer deployment only, one dashboard instance per printer	Future: fleet manager with N printer nodes and central dashboard

Conclusion: The system qualifies as a Level 3 Digital Twin (true Digital Twin, not Digital Shadow) in the Kritzinger taxonomy, because bidirectional data flow is implemented. However, autonomous actuation beyond print pause/abort is not yet available.

E.2 Technology Readiness Level (TRL) Assessment

TRL Level	Status	Evidence
TRL 1: Basic principles	Complete	Literature review (Chapter 1), FMEA (Chapter 2), physics-based thermal and vibration models
TRL 2: Technology concept	Complete	Digital twin architecture designed, five-layer model validated conceptually
TRL 3: Experimental proof	Complete	Feature engineering validated on real sensor data, ML models trained and evaluated
TRL 4: Lab validation	Complete	Three case studies conducted in university lab, 551,070 rows collected, 79.77% accuracy on holdout
TRL 5: Relevant environment	Partial	Tested only on BCN3D+ in one lab, not yet validated on different printer models or production settings
TRL 6: Prototype demo	Partial	Working prototype demonstrated, dashboard functional, label application submitted for external review
TRL 7+: System prototype	Future	Multi-printer deployment, long-term field study, commercial packaging not yet done

E.3 Known Limitations and Mitigations

Limitation	Impact	Severity	Proposed Mitigation
Single printer dataset (BCN3D+, n=11 sessions)	Models may not generalize to other FDM models or filament types	Medium	Multi-printer data collection campaign, transfer learning across printer models
Catastrophic class absence in holdout (clog, loose_belt, heat_creep)	Cannot report precision/recall for worst-case faults	High	Dedicated fault injection sessions with shorter prints, sliding window split instead of temporal holdout
Session-level labels (not row-level)	Some rows labelled 'heat_creep' may be healthy, noisy labels reduce model quality	Medium	Automated real-time labelling triggered by Tier 0 events, OctoPrint event logging
Single sensor node (no redundancy)	Sensor failure causes complete monitoring blackout	Low-Medium	Watchdog timer on ESP8266, dashboard heartbeat monitor with 'sensor offline' alert
SQLite for persistence	Maximum ~5 concurrent readers, not suitable for fleet (N > 5 printers)	Low (single printer)	PostgreSQL migration planned for fleet version, SQLite WAL mode sufficient for 1 printer

Limitation	Impact	Severity	Proposed Mitigation
No vision-based monitoring	Layer delamination and geometric defects not detected	Medium	Camera module (Pi Camera 3) planned as optional Tier 3 extension using CNN defect detection

Conclusion

This appendix document provides the technical verification, experimental logs, and architectural specifications for a Hybrid Digital Twin for Predictive Maintenance, validating a system that is highly attainable, technically proficient, and clearly positioned for future expansion.

- ➔ Attainability: By utilizing low-cost, off-the-shelf components (such as the ESP8266, thermistors, and repurposed mouse encoders) paired with open-source frameworks (React 19, Three.js, and Python), the system proves that industrial-grade digital twin concepts can be successfully developed and deployed within strict budgetary and local resource constraints.
- ➔ System Capability: In terms of performance, the architecture demonstrates significant maturity. Achieving a Level 3 Digital Twin status under the Kritzinger taxonomy via bidirectional data flow, the proposed Physics-Aware Binary Ensemble reaches a 79.77% overall accuracy. More importantly, it achieves a high recall for progressive faults (up to 69.76% for kinematic degradation), proving its practical utility as an early-warning system that can autonomously trigger print-pausing via the OctoPrint API to prevent catastrophic failures.
- ➔ Data Limitations and Future Horizons: Despite these successes, the system's primary bottleneck remains its data constraint. Restructured around a single-printer dataset (n=11 sessions) with session-level rather than row-level labels, the models lack exposure to cross-model variation and acute, catastrophic fault data (such as complete clogs or sudden heat creep).

Ultimately, while bounded by current data availability, this work establishes a robust and scalable foundation. Future iterations expanding into multi-printer data campaigns, automated row-level labeling, and vision-based defect detection will easily elevate this framework from a functional prototype to a highly resilient, enterprise-ready manufacturing solution.